



THE UNIVERSITY *of* EDINBURGH
Edinburgh Medical School

Master of Data Science for Health and Social Care (online)

Dissertation

2026

B248593

***Detecting Movement Imbalances using custom
TensorFlow Lite Models utilizing the Google
MediaPipe framework.***

25000

Table of contents

Contribution	3
Abstract	4
Background	4
Aim & objectives	4
Methods	4
Results	4
Conclusion	5
1 Introduction	6
1.1 Research question	7
1.2 Aim	7
1.3 Objectives	7
1.4 Dissertation structure	7
2 Literature Review	9
2.1 Market research on motion capture systems	10
3 Methodology	12
3.1 Study design and setting	12
3.2 Data source and characteristics	12
3.3 Variables, features, and preprocessing	12
3.3.1 The classes	12
3.3.2 Primary outcome	13
3.3.2.1 Exposures/independent variables	13
3.3.2.2 Covariates/control features	13
3.3.2.3 Extracted features	14
3.4 Model development and transfer learning	14
3.5 Analysis and performance evaluation	14
4 Implementation	15
4.1 Research setup	15
4.1.1 Hardware and runtime	15
4.1.2 Software/framework/service stack	15
4.1.3 Environment manifests used in this project (congested by Dockerfiles)	15
4.1.4 Example reproducible local environment	16
4.1.5 Streamlit Web App Interface	16
4.2 Training pipeline and data engineering	17
4.2.1 Augmentation & split policy	17
4.3 Research clients, Inference pipeline, containerization and CI/CD	17
4.3.1 Containerization and compose	18
4.3.2 CI/CD	18
4.3.3 Automation orchestration (n8n)	18
5 Results	20
5.1 Dataset and preprocessing outcomes	20
5.2 2-Class Training	20
5.2.1 Model performance - 2-class problem	20
5.2.1.1 Training 1	21
5.2.1.2 Training 2	21

5.2.1.3	Training 3 (extra body landmark overlay)	24
5.2.2	Error analysis	24
5.3	3-Class Training	26
5.3.1	Model performance - 3-class problem	27
5.3.1.1	Training 1 (MobileNet-V2 architecture)	27
5.3.1.2	Training 2 (EfficientNet-Lite0 architecture)	29
5.3.1.3	Training 3 (EfficientNet-Lite2 architecture)	29
5.3.1.4	Training 4 (EfficientNet-Lite4 architecture)	31
5.3.2	Error analysis	33
5.3.2.1	Common failure modes observed in confusion matrices	34
5.3.2.2	Comparison of results to 2-class training	34
6	Discussion	35
6.1	Interpretation of findings	35
6.1.1	Implications and next steps	35
6.2	Strengths and limitations	35
6.3	Clinical and practical implications	36
7	Conclusion and Future Work	37
7.1	Clinical data collection	37
7.2	Expert annotation	37
7.3	Retraining the 3-class model	37
7.4	Evaluation and fairness	38
7.5	Deployment readiness	38
7.6	Milestones	38
8	Appendix A. Figures	39
	References	40

Contribution

The development of this dissertation incorporated the selective use of generative artificial intelligence (AI) tools as supportive research and technical assistance instruments. Github Copilot and Posit Assistant were utilized to assist in troubleshooting Python dependency conflicts, resolving CI/CD configuration issues involving Jenkins and Docker, and supporting administrative setup tasks related to RStudio Server. In addition, Perplexity was used for targeted literature discovery and contextual identification of relevant academic sources and references.

All critical analysis, interpretation, methodological decisions, and written academic content were independently evaluated, verified, and finalized by the author. This was achieved through systematic inspection of model outputs, including accuracy, loss trajectories, precision/recall metrics, and confusion matrices across training runs, as well as consistency checks between expected and observed model behavior.

Also, I received guidance and support from my two supervisor, Dr. Syed Ahmar Shah and John Wilson, who gave me many recommendations during the whole process to conduct the project as well reviewed the draft of the paper once and gave me valuable feedback.

Abstract

Background

Postural abnormalities and impaired motor coordination are associated with musculoskeletal disorder, whilst their assessment in clinical practice often relies on subjective observations and the clinician’s expertise. At the same time, increasing pressure on healthcare systems highlights the need for scalable, cost-effective and objective screening tools that can also be used outside highly specialised laboratory settings. Recent advances in deep learning have enabled automated posture and pose analysis. However, existing research focuses predominantly on general pose estimation or biomechanical tracking, whilst comparatively few studies have investigated clinically grounded models for posture classification based on existing physiotherapy frameworks, such as the Klein-Vogelbach concept of normal, curved (hypotonic) and tense (hypertonic) postural patterns. This study therefore investigates whether transfer learning on compact CNN architectures can support practical, cost-effective and device-based posture screening through the classification of clinically relevant postural states.

Aim & objectives

Finetuning and evaluation of a deep learning model for posture and movement quality assessment that detects and interprets imbalances in the head, neck torso to finetune an pre-trained Google MediaPipe image classification model for posture and movement imbalance detection; and to evaluate the performance of the finetuned classification model.

Methods

Quantitative, observational secondary data analysis evaluating transfer learning on lightweight convolutional neural networks (CNNs) using a curated Kaggle image dataset (n=300) according to clinically informed criteria derived from the physiotherapeutic Klein-Vogelbach framework to yield approximately 1,900 training images with validation and test splits. Transfer learning fine-tuned classification layers on EfficientNet-Lite0/2/4 and MobileNet-V2 backbones; preprocessing included resizing/normalization and clinically conservative augmentations. Models were exported fully as well as quantized TFLite artifacts and evaluated using accuracy, precision, recall, F1, confusion matrices, and balanced accuracy. A binary “normal vs non-normal” task was also trained to probe clinical interpretability and potential performance gains.

Results

Across the two-class experiments, the binary classification task achieved a balanced accuracy of 0.552 (with landmark overlays) and 0.536 (without overlays), compared to a 0.50 chance level.

Reformulating as a three-class experiment the EfficientNet-Lite4 model architecture achieved the highest balanced test accuracy (**41.7%**), followed by EfficientNet-Lite2 (**39.8%**). Confusion patterns showed over-prediction of “slumped” and frequent misclassification of “normal,” consistent with class overlap and subtle boundary cases. This approach indicates modest gains against 0.33 chance level and clearer clinical framing.

Conclusion

Transfer learning on compact CNNs within the used MediaPipe stack is feasible for posture imbalance screening, but current performance is limited potentially constrained by a small sample size, class ambiguity, and domain subtlety. Therefore targeted data growth (especially boundary and “normal” exemplars), refined labels, posture-aware augmentation, calibration/thresholding, and fairness checks are recommended.

1 Introduction

Liu et al. (2025) states that there is an increasing pressure on healthcare systems through musculoskeletal disorders with projected rise to 2.161 billion people in 2035. Therefore, healthcare is progressively recognizing the importance of movement quality and postural balance in preventing injury, managing chronic conditions, and optimizing physical performance (Whittaker et al., 2017). Wijekulasuriya et al. (2025) found in an extensive recent meta analysis that traditional methods for assessing posture and movement often rely on subjective clinical observations, limiting their accessibility and consistency. Deep learning and computer vision offer promising avenues for developing such objective tools that are scalable and run on consumer hardware that is widely available (Bazarevsky & Grishchenko, 2020; Google, 2024). This dissertation explores the development of a deep Learning algorithm for supporting the objective assessment of human posture.

Deep learning is a subset of machine learning based on multi-layered artificial neural networks that learn hierarchical feature representations directly from raw data such as images or motion sequences. Unlike traditional approaches that rely on manually engineered features, deep learning models automatically extract complex, non-linear patterns like edges in images. This enables state-of-the-art performance in domains such as monitoring physical activities of patients . Transfer learning is used to simplify adoption, allowing the reuse of learned visual representations for domain-specific posture classification, extending this paradigm by reusing knowledge from pretrained models. These models often learned from large-scale datasets with wider focus. So, instead of training models from scratch, pretrained networks (e.g., image classification or pose estimation models) are fine-tuned on domain-specific data such as clinical posture datasets. This significantly reduces data requirements and improves generalization, particularly in settings where labelled clinical data are limited or expensive to obtain (Ray & Kolekar, 2024).

This thesis employs a deep learning method—specifically, a transfer learning approach—to develop a resource-efficient, operational system for posture analysis that can run on standard hardware. Pretrained feature extractors are adapted to classify clinically defined posture categories, leveraging prior knowledge from large-scale vision models to compensate for limited domain-specific data. This combination enables a transition from subjective visual assessment toward scalable, data-driven posture evaluation suitable for healthcare screening, rehabilitation, and movement analysis applications.

Human posture and coordination are central to both health and performance. Impairments such as muscular tension, poor motor control, or postural imbalance contribute significantly to discomfort, injury, and reduced physical performance in daily and professional life (Kubalek-Schröder & Dehler, 2013). Advanced motor skills rely on precise kinaesthetic differentiation, including fine motor control, proprioceptive sensitivity, accurate force regulation, coordinated timing, and the economy of movement. These elements form the basis of effective movement in sports, rehabilitation, and everyday function (Zaucker, 2010). Recent research highlights that chronic low back pain and other movement system impairments are closely associated with sustained postural deviations and maladaptive movement patterns that often develop gradually and may not be apparent during isolated clinical assessments (Ge et al., 2022; Sahrman et al., 2017).

Traditional approaches to assessing posture and coordination rely on subjective evaluations, clinical observation, expensive motion analysis systems, or extensive practitioner expertise. While commercial software for posture assessment exists, these solutions often require professional oversight and remain limited in their ability to capture subtle or habitual patterns of imbalance consistently (Badhe & Kulkarni, 2018). This highlights a clear gap between the qualitative

nature of clinical assessments and the potential for more objective, data-driven methods enabled by modern deep learning techniques.

Recent advances in computer vision and deep learning algorithms provide opportunities to bridge these limitations. Convolutional Neural Networks (CNNs) have demonstrated high accuracy in identifying patterns within image and video data and are increasingly applied in gait analysis, posture detection, and motion tracking (Jiang et al., 2023). Among available frameworks, the Google MediaPipe framework brings a lightweight approach that can be run in the browser or on mobile devices, utilizing a statistical 3D human shape modelling pipeline offered as a trainable and modular deep learning framework. It can accurately represent full-body details to detect pose and posture (Bazarevsky & Grishchenko, 2020; Xu et al., 2020).

As a physiotherapist with focus on movement quality, I found these gaps were not purely academic but deeply practical in my clinical practice. To reach a level of good understanding of movement (patterns) it required years of training and clinical experience to develop. For instance the ability to accurately assess the quality of movement based on key indicators such as the passive rotatory mobility of the fifth thoracic vertebra (Th5) and adjacent vertebrae (Tachihara & Hamada, 2019).

1.1 Research question

How can transfer learning on a pre-existing TensorFlow Lite model for pose estimation and image classification be used to detect and analyze posture and movement imbalances?

1.2 Aim

Finetuning and evaluation of a deep learning model for posture and movement quality assessment that detects and interprets imbalances in the head, neck torso and to make it suitable for the clinical context.

1.3 Objectives

1. To finetune an pre-trained Google MediaPipe image classification model for posture and movement imbalance classification.
2. To evaluate the performance of the finetuned classification model.

1.4 Dissertation structure

The present work is structured as follows:

The abstract formulates the research question, aim, objectives, and provides a concise summary of the design, methods, results, and conclusions. Where as the introduction outlines the background, rationale, and research question, and defines the aim and objectives of the project.

The “Literature Review” chapter provides a comprehensive literature review of the current state of AI-based posture and movement analysis, with a focus on the capabilities and limitations of existing pose estimation frameworks such as Google MediaPipe.

The “Methodology” chapter outlines the methods used, including the study design, data sources, variable definitions, model development approach, and performance evaluation strategy.

The “Implementation” chapter details the implementation of the training and inference pipelines with a complex CD/CI integration.

The “Result” chapter presents the results of model performance and error analysis and shows the challenges of training a custom CNN model.

The “Discussion” chapter addresses the findings in relation to existing literature, strengths and limitations, and clinical implications.

The “Conclusion” chapter outlines the expected impact and dissemination plans, while

Whereby the final chapter concludes with a summary of contributions and future work directions.

2 Literature Review

Cao et al. (2024) describe the detection of human postures undertaken by artificial intelligence as a process in which machines mimic the human visual system and learning behaviour by extracting distinctive features from sensor or image data and subsequently using these features to classify or track postures over time. Furthermore they divide into static and dynamic posture recognition, whereas the static posture recognition treats each input (image or single frame) as a standalone instance. Models for these purposes are trained and perform inference on individual frames, using image-based classifiers such as scale-invariant feature transform, histogram of oriented gradients, support vector machine (SVM), Gaussian mixture model, dynamic time warping, hidden Markov model (HMM), lightweight network and convolutional neural networks (CNN) (Jiang et al., 2023). Cao et al. (2024) describe the dynamic pose detection as processing a continuous image sequence (video), with models capturing temporal evolution via recurrent structures such as BiGRUs or similar time-aware networks. As they explain, CNNs perform this capturing or, more precisely, feature extraction through the use of a deep convolutional neural network that locates skeletal keypoints and derives a multidimensional set of pose features, much like the human eye recognises edges, shapes and joints. The extracted pose features are then learned as spatial features (body configuration in each individual frame) and temporal features (how the pose changes across successive frames). These models are also called Pose Estimation Models (PEMs). A widely used open-source example of a CNN PEM is Google’s MediaPipe framework for real-time pose estimation (Lugaresi et al., 2019).

Recent validation studies comparing MediaPipe with gold-standard motion capture systems reveal measurable discrepancies in landmark accuracy, particularly for subtle postural features, highlighting the need for domain-specific refinement for clinical use (Kakuta et al., 2022). Here, “subtle” means inappropriate muscular tension with small or no movement of (the) body(-parts). More broadly, current pose estimation models remain limited by sensitivity to noise and 2D–3D estimation gaps, reducing their reliability for detecting the named tensions and asymmetries critical to movement and pose imbalance analysis (Kakuta et al., 2022; Lachance et al., 2023). Fairness and representativeness in pose estimation systems are also important concerns: current PEMs exhibit performance disparities across demographic groups, with reduced accuracy for under-represented populations in training datasets (Lachance et al., 2023). Available datasets frequently focus on male and predominantly white participants between the ages of 19 and 50, while under-representing females, older adults, and individuals with darker skin tones, thereby increasing the risk of systematic bias and reduced generalizability in movement and posture analysis.”

Consequently, while MediaPipe Pose serves as a robust foundation for pose detection, in its standard configuration of training it does not inherently capture the fine-grained symmetry metrics, sagittal and coronal balance indicators, or compensatory movement patterns that rely on hyper- and hypo-tension that are essential for clinical postural and even more for movement analysis. Computer-vision-based deep learning markerless systems nonetheless represent an important step towards closing the gap between theoretical research, that can be even a century old like the principle of primary control of human movement by F.M. Alexander (2017), and practical implementation in remote clinical assessment (Ultralytics, 2025; Wagh et al., 2024).

This project uses transfer learning (Kakuta et al., 2022) to adapt a pre-trained Google MediaPipe CNN model, originally trained on extensive image data, to a new small Kaggle dataset but with more relevant and higher-quality images for posture imbalance detection. By utilizing pre-learned feature representations, the model can achieve faster convergence and improved accuracy even with a comparatively smaller or domain-specific dataset.

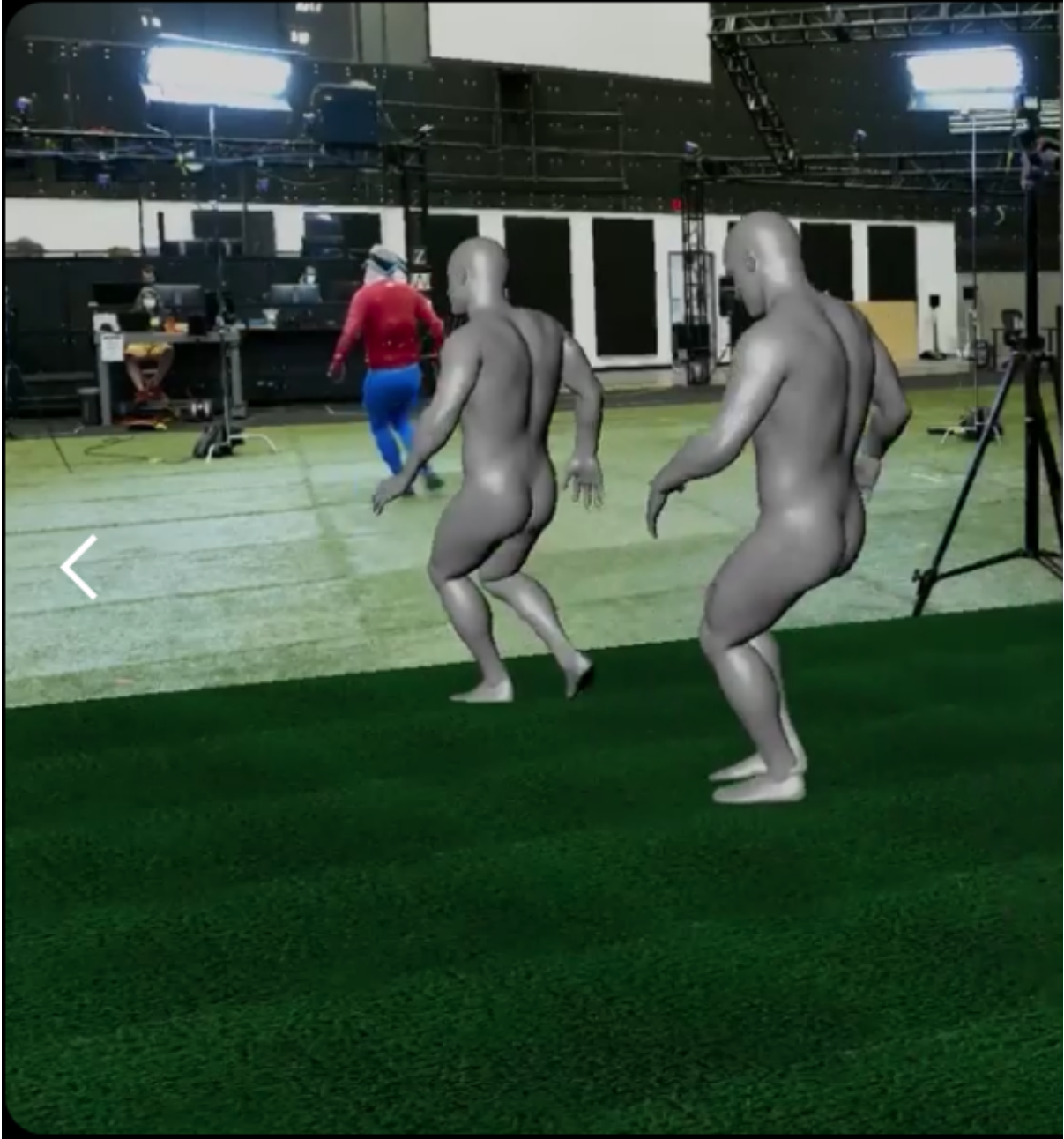
2.1 Market research on motion capture systems

Modern video analysis with the purpose of detecting pose, movement, and specific markers in sports activities has undergone a paradigm shift, moving from subjective clinical intuition to high-fidelity, data-driven kinematic quantification. Commercial ecosystems like RUNRIGHT 3D (2026) leverage dual-camera 3D reconstruction to analyze over 40 biomechanical metrics without intrusive markers, targeting retail efficiency and rapid, evidence-based footwear recommendations. Similarly, RunDNA (2026) employs advanced motion capture to track kinetic and kinematic variables from foot strike to toe-off, providing practitioners with a granular, 3D anatomical breakdown of movement asymmetries. For integrated clinical environments, the TecnoBody Walker View (2026) combines 3D camera technology with a sensorized, load-cell-equipped treadmill to provide real-time biofeedback and synchronized gait analysis. Supplementing these laboratory-grade tools, ViMove2 (2026) utilizes wearable inertial measurement units (IMUs) to provide continuous, longitudinal monitoring of temporal outcomes like ground contact time and cadence, albeit with sensitivity to speed-dependent discrepancies. At the vanguard of performance capture, Move AI (2026) offers sophisticated, markerless motion capture solutions that utilize computer vision to extract high-fidelity skeletal data from multi-camera footage, facilitating biomechanical analysis outside of traditional studio settings as shown in Figure 1.

The overarching advantage of these commercial approaches lies in their ability to translate complex, high-frequency spatial-temporal data into actionable clinical diagnostics. They reduce inter-rater variability, the differences between multiple assessors, and enable objective longitudinal tracking. Such standardization of classification, may improve the reliability of qualitative (and quantitative) movement assessments in evidence-based rehabilitation settings. Conversely, the disadvantages often involve high capital expenditure, the requirement for controlled capture environments, and often a reliance on proprietary pipelines that can be difficult to consistently integrate into existing clinical (analytical) workflows.

When evaluated against image classification models, the advantages of the latter become apparent: low-cost, low-latency identification of specific postural states and movement patterns, albeit at the expense of the quantitative kinematic precision provided by advanced motion-capture and biomechanical analysis systems.

Figure 1: Move AI advertising accurate markerless motion capturing on their website.



3 Methodology

In this work, deep learning techniques are applied through transfer learning on a pre-trained TensorFlow Lite model for posture and movement imbalance detection. The study involves secondary analysis of an existing image dataset curated for posture assessment, with a particular focus on evaluating the performance of a fine-tuned CNN model within the Google MediaPipe framework. This technology was selected because it provides a reproducible and cross-platform environment for rapid prototyping, model deployment, and iterative optimization of computer-vision applications, while enabling efficient integration of pre-existing perception components and consistent performance evaluation across different devices and platforms (Lugaresi et al., 2019).

3.1 Study design and setting

This study is a secondary data analysis exploring postural and movement imbalance detection using deep learning algorithms. The project applies a quantitative, observational design, analyzing existing image data through supervised deep learning, specifically Convolutional Neural Networks (CNNs) implemented via TensorFlow Lite within the Google MediaPipe framework. The aim is to evaluate whether lightweight AI models can detect subtle patterns of imbalance in posture and coordination and to adapt them for potential clinical use through transfer learning on a posture-specific dataset.

The study will be conducted remotely using a secondary dataset. All data analysis will be performed on secured infrastructure provided by the author (see Implementation). No new data collection or participant interaction is planned.

3.2 Data source and characteristics

This study uses secondary image data of an openly available “Posture Keypoints Detection – Photos & Labels” dataset on Kaggle. The dataset consists of 300 images in static side view with image resolutions of up to around 1.6 MP and an overall high quality look.

Unlike large-scale, general-purpose image datasets, this Kaggle dataset is specifically curated for posture analysis. It focuses on side-view images suitable for assessing sagittal-plane alignment and upper-body posture. The relatively small dataset size and potential demographic limitations may constrain generalizability and fairness.

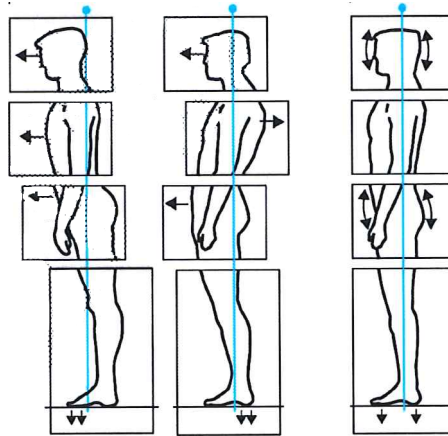
3.3 Variables, features, and preprocessing

The posture classification follows 3 types of poses regarding posture classifications by Klein-Vogelbach (1990) as illustrated in Figure 2.

3.3.1 The classes

- “tensed posture” (left)
- “slumped posture” (middle)
- “normal posture” (right)

Figure 2: 3 types of posture types defined by Klein-Vogelbach, the underlying functional movement concept was developed between 1955-1975.



The author manually annotated and assigned the 300 images to one of these classes according to his clinical experience in the field of orthopaedics and functional movement therapy. In the initial annotation round, the dataset was artificially balanced by assigning 100 images to each class (normal, slumped, and tensed posture). While this ensured equal class distribution for training, it also introduced ambiguous and potentially misclassified samples near class boundaries. Therefore, a second annotation and training run was conducted using a slightly imbalanced but clinically more consistent class distribution, resulting in improved class separation and overall classification performance.

3.3.2 Primary outcome

- **Head-neck-torso alignment:** Angular relationships and relative positioning of the head, cervical spine, and torso regarding the 3-class model by Klein-Vogelbach will lead towards model classification
 - normal posture
 - slumped posture
 - tensed posture

3.3.2.1 Exposures/independent variables

- **Image category/context:** Static posture images in standardized side view.
- **Camera/view angle:** Minor variation in pose estimation accuracy depending on exact lateral positioning.

3.3.2.2 Covariates/control features

- **Image characteristics:** Resolution, background, and lighting, which may influence model performance.
- **Posture variation:** Degree of anterior head translation, kyphosis, and other sagittal-plane deviations.

3.3.2.3 Extracted features

- **Skeletal keypoints:** Annotated landmarks in YOLO pose format and MediaPipe Pose outputs.
- **Kinematic proxies:** Angular relationships between key joints and segments.
- **Postural imbalance markers:** Anterior head translation, sagittal imbalance, and deviations from expected alignment patterns.

All selected images from the Kaggle dataset undergo data augmentation (e.g., small rotations, cropping, brightness adjustments) to increase sample size and improve generalization while preserving clinically meaningful posture characteristics.

3.4 Model development and transfer learning

This study employs a transfer learning approach to develop a domain-specific TensorFlow Lite model suitable for clinical posture and movement analysis. Transfer learning enables the adaptation of pre-trained CNN architectures—such as those used within MediaPipe and trained on large-scale datasets like ImageNet—to the focused task of detecting head-neck-torso imbalances. In practice, base model weights are kept frozen or partially frozen, while classification and selected high-level layers are retrained using the Kaggle posture dataset.

Deploying the resulting model as a quantized TensorFlow Lite artifact supports real-time, on-device inference on mobile and edge devices, reducing latency and preserving user privacy by processing data locally.

3.5 Analysis and performance evaluation

The analysis will proceed in two phases:

1. Descriptive analysis of dataset composition, annotation completeness, and posture-related metrics.
2. AI-based classification and pattern recognition using transfer learning and supervised deep learning methods.

Model performance will be evaluated using accuracy as the primary metric and, where feasible, precision, recall, F1-score, and confusion matrices. Robustness and generalization will be explored through limited distribution-shift experiments (e.g., withheld images with slight background or lighting variation). All hyperparameters, data splits, and preprocessing steps will be documented to ensure reproducibility.

4 Implementation

To utilize the Google MediaPipe framework, an extended pipeline and automation were used to prepare the data, train and evaluate models, and produce deployable artifacts. The research environment and reproducible steps to build the training pipeline and maintain a production ready data-engineering workflow (including the n8n augmentation workflow) are described in this chapter.

4.1 Research setup

4.1.1 Hardware and runtime

The development was mostly performed on a remote virtual machine with Ubuntu Linux and specs of 8GB RAM and 4 available CPU Cores. For editing Python scripts and notebooks, the Jupyter Notebook was used. The transfer learning script was usually run for training a custom model locally on a MacBook Pro due to its superior performance.

For editing the Quarto dissertation project file a RStudio Server was utilized. Next to this, a Streamlit WebApp was developed and used to test re-trained models manually. Both the Jupyter notebook and the Streamlit WebApp were deployed via a Docker container on the remote server to manage heavy Python library dependencies of MediaPipe Model Maker (Python 3.10) respective of MediaPipe / TensorFlow Lite (Python 3.13) for the Streamlit app.

4.1.2 Software/framework/service stack

Category	Technologies / Services
Conda programming environments	Python 3.10 (Model Maker), Python 3.13 (Inference)
Core deep learning frameworks	MediaPipe Pose 0.10.20, MediaPipe Model Maker 0.2.1.3, TensorFlow (+Lite, bundled) 2.19.1, Keras 3.12.1
Supporting Python libraries	OpenCV, NumPy, pandas, Matplotlib, FastAPI
Development and authoring tools	RStudio Server, Jupyter Notebook, Quarto
Deployment and containerized services	Docker, Streamlit WebApp, MediaPipe API
CI/CD and automation	Jenkins, GitHub Actions, n8n

4.1.3 Environment manifests used in this project (congested by Dockerfiles)

code/

environment.yml

environment_model-maker.yml

requirements.txt

requirements_model-maker.txt

4.1.4 Example reproducible local environment

All scripts, notebooks and the web app can also be run locally; to do this, a Conda environment needs to be created:

```
# MediaPipe Model Maker environment
conda env create -f code/environment_model-maker.yml
conda activate dissertation-3.10

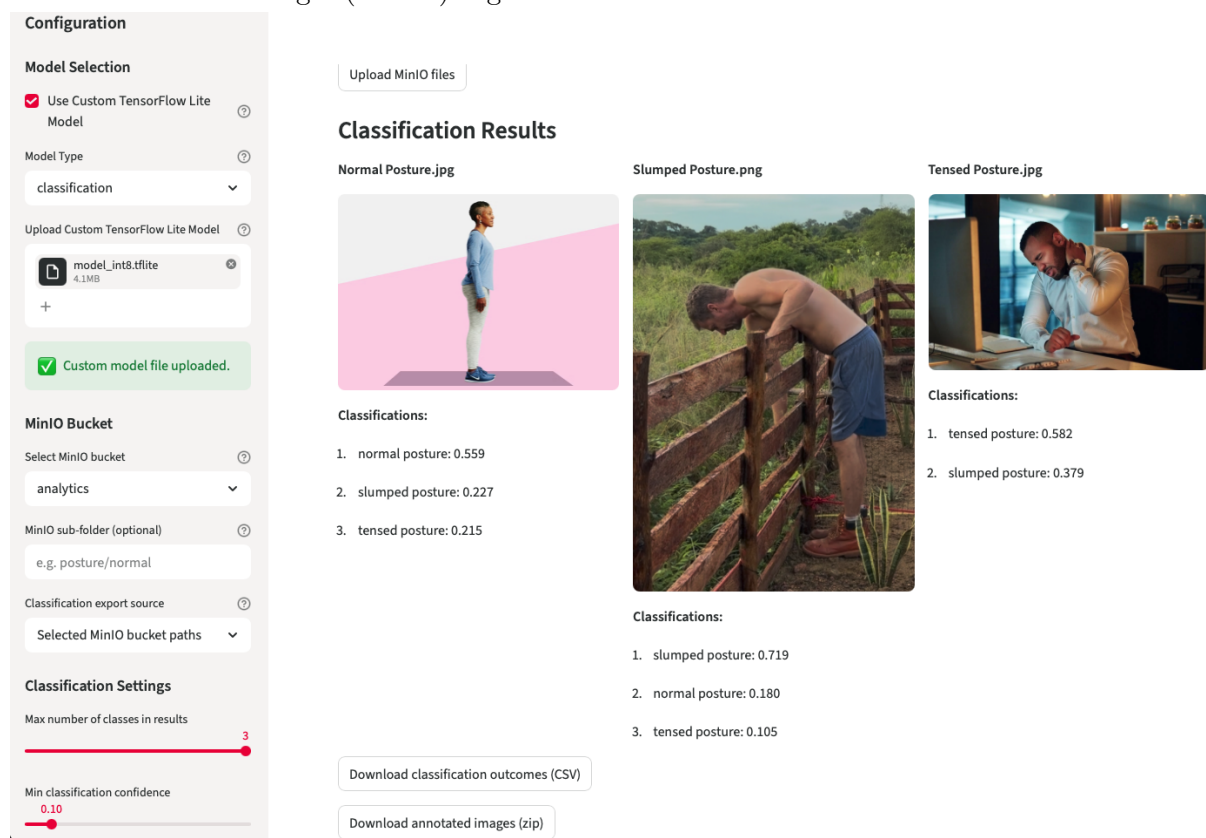
# MediaPipe Pose and TensorFlow Lite environment
conda env create -f code/environment.yml
conda activate dissertation-3.13
```

4.1.5 Streamlit Web App Interface

To utilize the existing MediaPipe Pose model to create images with the YOLO 33 body landmarker overlay, for testing purposes an Interface was developed. Therefore Streamlit, an open-source Python framework was chosen to build a graphical user interface at `code/mediapipe_pose.py`.

Streamlit is widely used in the data science sphere, to build interactive data apps. The same Interface was used to test the build classification models, as you can see in Figure 3.

Figure 3: Streamlit Web App with classification of postures using the build quantized model from Training 4 (3-class) Figure 11.



The app can digest images from the Browser as User upload and from the self-hosted MinIO server. As later described, a n8n automation was processing images automatically from a MinIO

bucket (and saved it back to a second bucket), the App uses the same API to classify or annotate images (when a pose model is chosen). This led to a more efficient development, as only one source of truth for the image detection/processing code was needed.

4.2 Training pipeline and data engineering

Overview - The training pipeline follows an ETL pattern: ingest → validate → augment → split → train → evaluate → archive. Steps are implemented in the repository as Jupyter notebooks and scripts to allow both interactive experimentation and non-interactive CI execution.

Automated augmentation (n8n + MediaPipe API) - To expand the original 300 images, the repository uses an n8n workflow to produce duplicates with MediaPipe keypoint overlays and controlled image edits (rotations, crops, brightness adjustments). - The exported n8n workflow is available at `code/n8n/Dissertation_Posture_Analysis.json`. The high-level workflow:

1. Read a list of source images (the initial 300).
2. For each image, call a MediaPipe inference endpoint (either the containerized FastAPI MediaPipe API) using an HTTP Request node.
3. Receive keypoint coordinates and render overlay images server-side.
4. Save augmented images to MinIO bucket.

Training execution - Training can be run locally via the hosted Jupyter notebook on `custom_image_classifier_model_training.ipynb`.

4.2.1 Augmentation & split policy

Augmentations used brightness/contrast jitter, cropping, tilting, small rotations, and MediaPipe body landmark overlays. Splits are deterministic and saved into sub-folders in `data/` for reproducibility.

Artifacts and model storage - Model artifacts (.h5 and .tflite), logs and run figures are stored under `data/models/` and `runs/`. Each run directory includes metadata.

4.3 Research clients, Inference pipeline, containerization and CI/CD

Inference research client scripts live in `code/scripts/` (for example `custom_tflite_image_classifier.py` and `custom_tflite_pose.py`) and operate against quantized .tflite models exported to `data/models/`.

Local research client example:

```
python code/scripts/custom_tflite_image_classifier.py --model data/models/<run>/model.tflite
```

As a counterpart to the Streamlit Web App there is also a command line tool for testing (also utilizing the API). Example:

```
python code/scripts/human_posture_analysis.py --mode image --api-base-url http://seriousben
```

4.3.1 Containerization and compose

The project maintains two primary Dockerfile artifacts:

`code/`

`Dockerfile` — training and utilities image.

`Dockerfile_mediapipe`

MediaPipe runtime used for keypoint extraction and overlay rendering. Multi-service development is defined in `code/docker-compose.yml`.

To start development services docker compose was used:

```
docker compose -f code/docker-compose.yml up --build
```

4.3.2 CI/CD

Jenkins is used for build a Jupyter environment on a server to run model training and also builds a Streamlit WebApp for inference testing of previously build models.

Github Actions of a Github Pages repository is triggered to render the Quarto artifacts:

- Checkout repository - Installs Quarto and LaTeX (for PDF rendering).
- Deploys the dissertation project as HTML on <https://drbenjamin.github.io/dissertation.html>

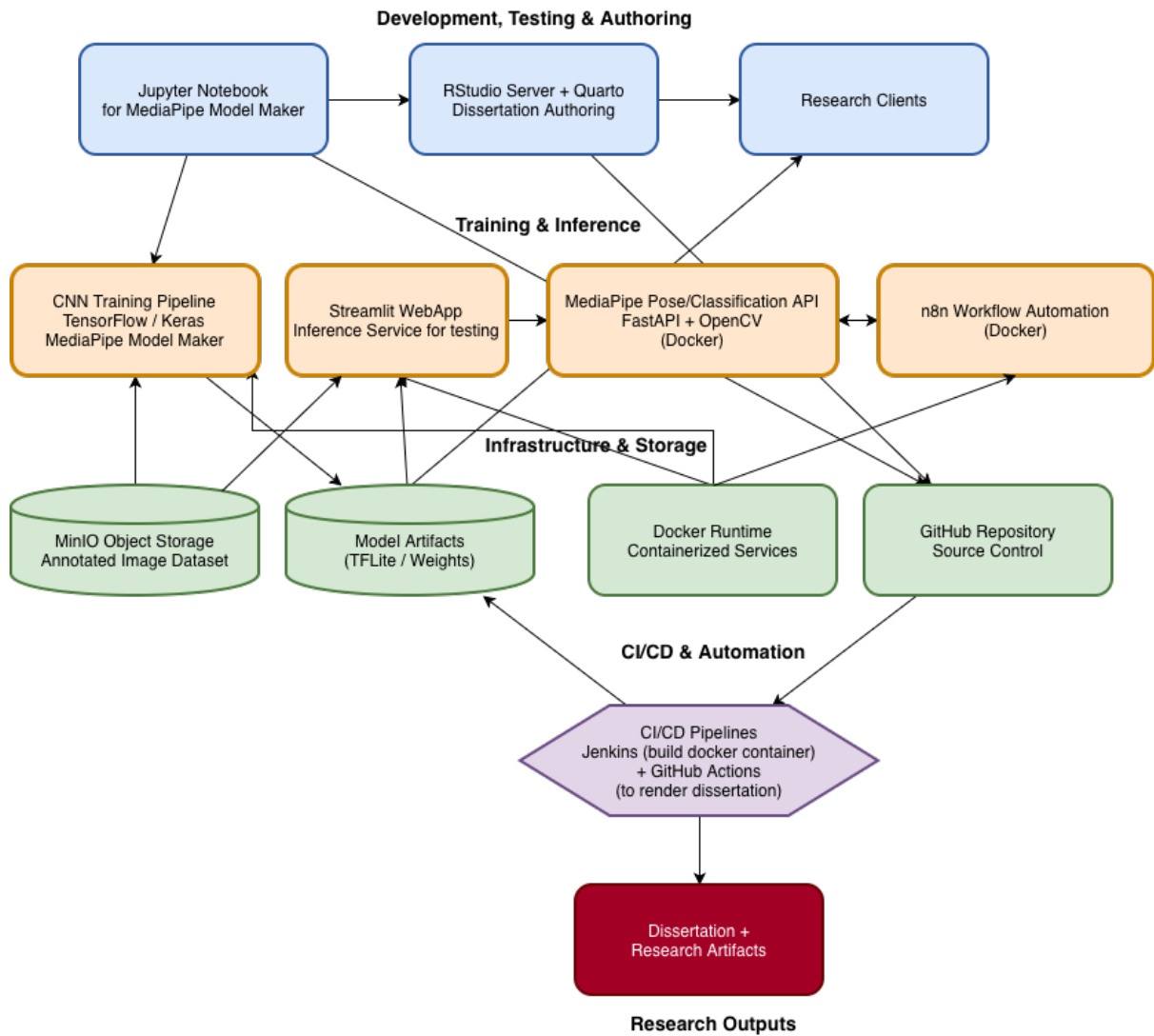
4.3.3 Automation orchestration (n8n)

A `n8n` server run as a containerized service (two container) to hosts the augmentation workflows. Responsibilities:

- `n8n` Canvas for editing the workflows (container one)
- MediaPipe API (container two) for inference calls.
- MinIO file storage to persist augmented images.

The Figure 4 illustration shows the whole implementation setup and dependencies.

Figure 4: Implementation map illustrates the pipelines, automation and containerization.



5 Results

Results were measured for seven model trainings in total. Three for 2-class and 4 for 3-class image input datasets.

5.1 Dataset and preprocessing outcomes

The dataset and splits used across all runs were consistent. The training set contained 300 original images distributed across two or three classes. To increase the number from 300 original images to counts of ~ 1500 or ~ 1800 , different augmentation techniques were applied.

- First, the images were duplicated several times and processed via random image editing utilizing the OpenCV framework. For instance, the color-theme or background color were changed, images were cropped or tilted. For each augmented image, 2 changes were applied.
- Second, a subset of duplicated 300 images with body landmarks as overlay were added to specific runs. This was achieved by an existing MediaPipe Pose model which was adding these annotations.

Preprocessing was applied consistently for all runs (same processed image data set) to maintain comparability between 2/3 class runs and architectures.

5.2 2-Class Training

The objective is to reach an accuracy of approximately 80–90%, which is generally considered strong performance for practical detection tasks in applied deep learning settings and for the Google MediaPipe model architecture. Therefore, the images will be annotated into normal and non-normal to define 2 classes. As the first training run consisted of only 300 images in total, which were further divided into training, validation and testing subsets, the largest available architecture class, **EfficientNet-Lite4**, was selected to maximize feature extraction capacity despite limited dataset size. EfficientNet-Lite4 was also retained for training run 2 and 3, although dataset sizes increased substantially. This was done to maintain comparability across experiments.

5.2.1 Model performance - 2-class problem

The same base dataset was used for all three training runs. To address class imbalance in run 2 and 3, data augmentation was applied with an $8\times$ multiplication for normal posture images and a $4\times$ multiplication for non-normal posture images, resulting in a balanced dataset of 1,580 synthetically augmented images.

The third run was extended by an additional 300 images, which were augmented using body landmark overlays generated with the Google MediaPipe Pose framework to incorporate explicit skeletal structure information, which lead to a total of 1880 images.

5.2.1.1 Training 1

Prediction-derived overall accuracy: 0.6000

Model evaluate() loss/accuracy on dataset_obj: 0.546806/0.733333

Confusion matrix (rows=true, cols=pred)

True predicted	non-normal	normal
non-normal	16	8
normal	4	2

Row-normalized confusion matrix (recall by class)

True predicted	non-normal	normal
non-normal	0.6667	0.3333
normal	0.6667	0.3333

Per-class support:

- non-normal posture: 24
- normal posture: 6

Balanced accuracy (macro recall): **0.5000**

Classification report

Class	Precision	Recall	F1-score	Support
non-normal posture	0.8000	0.6667	0.7273	24
normal posture	0.2000	0.3333	0.2500	6
Accuracy			0.6000	30
macro avg.	0.5000	0.5000	0.4886	30
weighted avg.	0.6800	0.6000	0.6318	30

5.2.1.2 Training 2

Prediction-derived overall accuracy: 0.5316

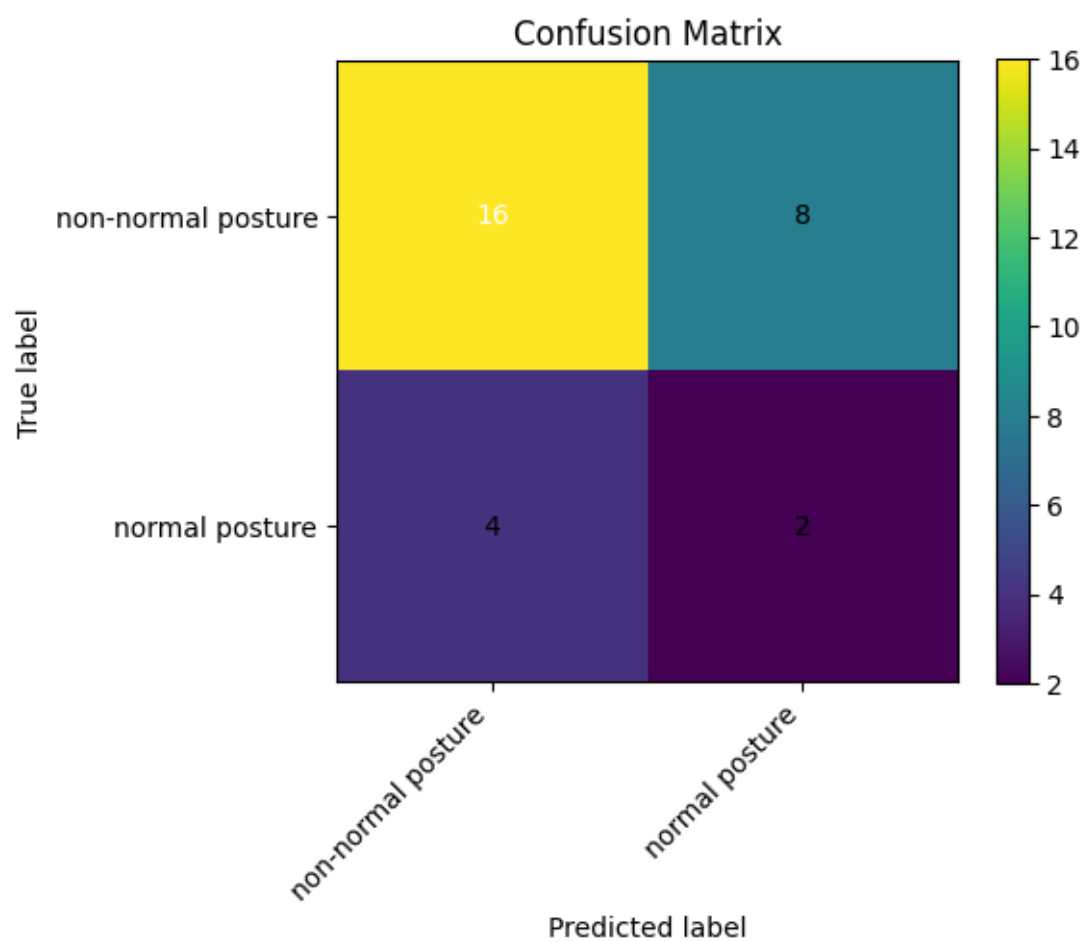
Model evaluate() loss/accuracy on dataset_obj: 0.421809/0.854430

Confusion matrix (rows=true, cols=pred)

True predicted	non-normal	normal
non-normal	35	47
normal	27	49

Row-normalized confusion matrix (recall by class)

Figure 5: 2-Class - Training 1 confusion matrix showing low amount of test images and preference for category 'non-normal posture'.



True predicted	non-normal	normal
non-normal	0.4268	0.5732
normal	0.3553	0.6447

Per-class support:

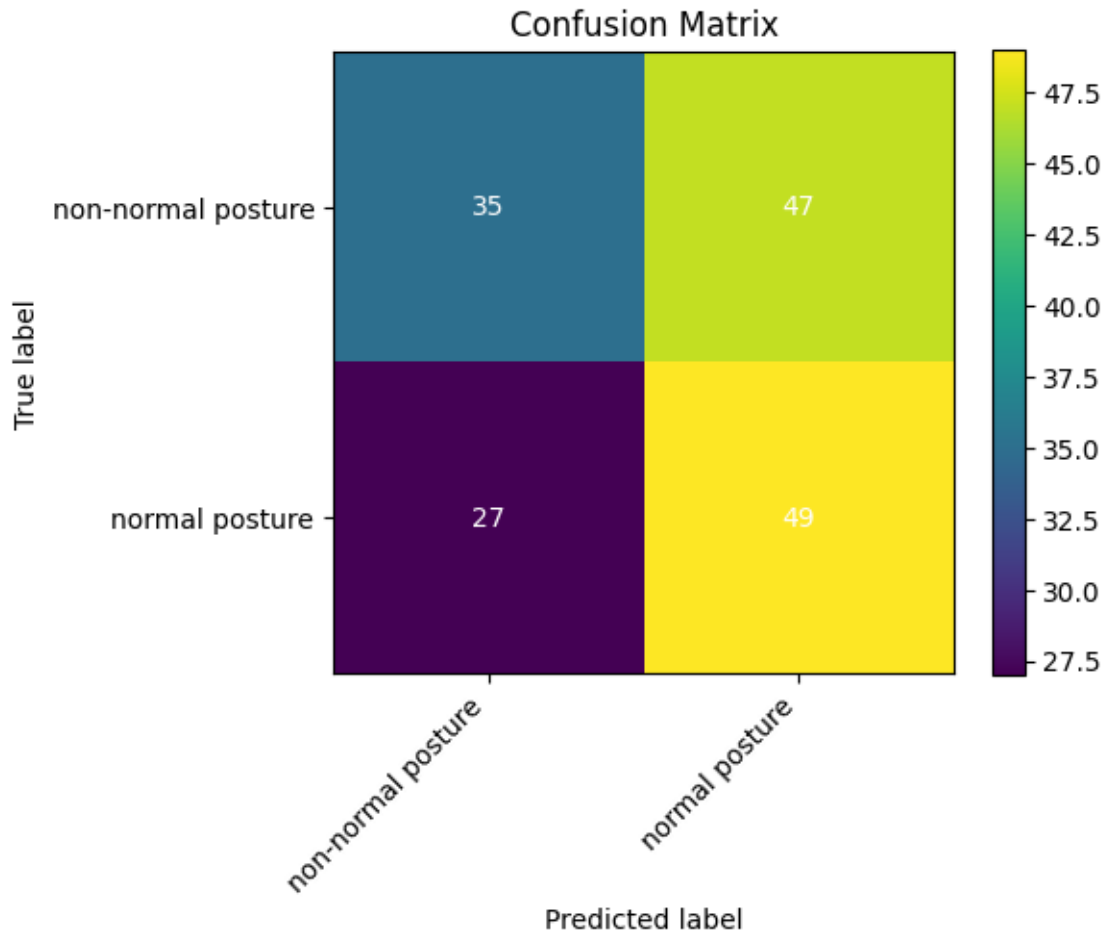
- non-normal posture: 82
- normal posture: 76

Balanced accuracy (macro recall): **0.5358**

Classification report

Class	Precision	Recall	F1-score	Support
non-normal posture	0.5645	0.4268	0.4861	82
normal posture	0.5104	0.6447	0.5698	76
Accuracy			0.5316	158
macro avg.	0.5375	0.5358	0.5279	158
weighted avg.	0.5385	0.5316	0.5264	158

Figure 6: 2-Class - Training 2 confusion matrix showing ‘normal posture’ class preference.



5.2.1.3 Training 3 (extra body landmark overlay)

Prediction-derived overall accuracy: 0.5585

Model evaluate() loss/accuracy on dataset_obj: 0.454466/0.840426

Confusion matrix (rows=true, cols=pred)

True predicted	non-normal	normal
non-normal	44	47
normal	36	61

Row-normalized confusion matrix (recall by class)

True predicted	non-normal	normal
non-normal	0.4835	0.5165
normal	0.3711	0.6289

Per-class support:

- non-normal posture: 91
- normal posture: 97

Balanced accuracy (macro recall): **0.5562**

Classification report

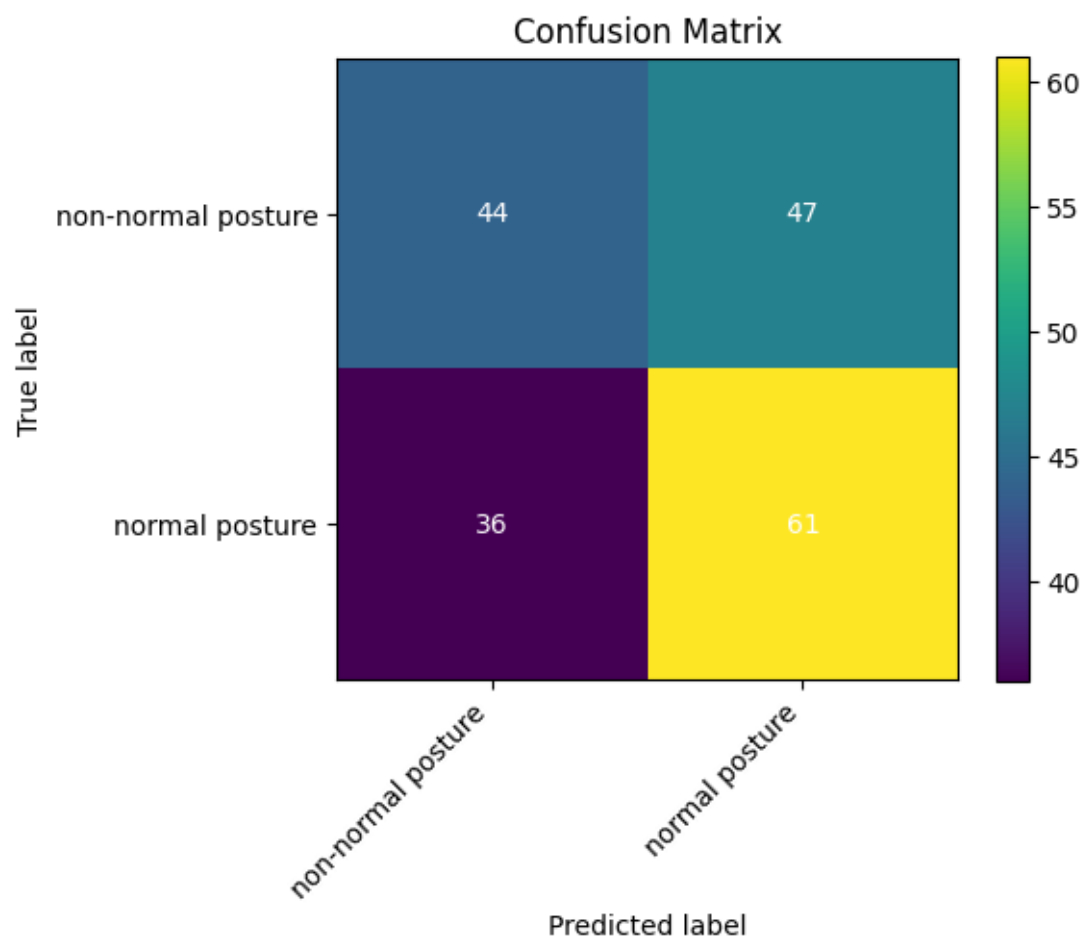
Class	Precision	Recall	F1-score	Support
non-normal posture	0.5500	0.4835	0.5146	91
normal posture	0.5648	0.6289	0.5951	97
Accuracy			0.5585	188
macro avg.	0.5574	0.5562	0.5549	188
weighted avg.	0.5576	0.5585	0.5562	188

5.2.2 Error analysis

Training run 1 achieves the highest accuracy of 0.6000. The confusion matrix using the test-image subset shows that performance is uneven across classes: out of 24 non-normal posture cases, 16 are correctly classified and 8 are misclassified as normal, resulting in a recall of 0.6667. In contrast, only 2 out of 6 normal posture cases are correctly identified, while 4 are misclassified as non-normal, yielding a recall of 0.3333. This indicates a clear bias towards correctly predicting non-normal posture while under-detecting normal posture.

As the normal posture class contains only 6 test images, the recall estimate for this class is highly sensitive to single misclassifications and therefore unstable. Consequently, using accuracy as the primary performance indicator is misleading due to both the small sample size and the strong class-wise imbalance. Therefore, **balanced accuracy (macro recall)** is introduced as the primary evaluation metric, which is **0.5000**, as it assigns equal weight to each class and directly reflects class-wise sensitivity (recall), providing a more reliable assessment of model performance across posture categories. Secondly two annotation strategies to increase the total number of images are used for following runs:

Figure 7: 2-Class - Training 3 confusion matrix again showing ‘normal posture’ class preference.



- **noise:** Several OpenCV image processing routines were applied to multiply the original 300 images to changed duplicates. Techniques used are:
 - Random flip (Horizontal or vertical flipping)
 - Affine transform (Rotating, scaling, translating, and shearing)
 - Random crop + resize (Cropping a random window then resizing back to original size)
 - HSV jitter (Random hue, saturation, and value shifts in HSV space)
 - Intensity adjust (Changing contrast/brightness)
 - Gaussian noise (Additive Gaussian noise per pixel)
 - Random erasing (Erasing a random rectangle region)
- **overlay:** With a pre-trained Google MediaPipe Pose model, which estimates 33 body landmarks to create a skeletal pose representation, the original 300 images were processed into augmented copies with landmark overlays and color coding (white/red) to indicate posture angles and quality (see Fig. A1).

Training run 2 with no overlays but total 1500 images after annotation, achieves an accuracy of 0.5316 and a balanced accuracy of **0.5358**, slightly higher than the first Training run with only 300. The confusion matrix shows an uneven class-wise performance: out of 82 **non-normal posture** cases, 35 are correctly classified as non-normal, while 47 are misclassified as normal, resulting in a recall of 0.4268. In contrast, out of 76 **normal posture** cases, 49 are correctly identified, while 27 are misclassified as non-normal, yielding a recall of 0.6447.

Training run 3 with additional landmark overlay images (>1800 images) achieves an accuracy of 0.5585 and a balanced accuracy of **0.5562**. Compared to run 2, class-wise performance becomes more balanced: recall for non-normal cases decreases slightly, while recall for normal cases increases, reducing the class imbalance in sensitivity. This indicates that landmark overlays shift the decision boundary towards improved representation of the minority class, resulting in a more even distribution of classification errors across classes.

Overall, both ladder models still perform only marginally above the 50% chance level, but the overlay-based model provides a measurable improvement in class-balanced performance rather than accuracy alone.

5.3 3-Class Training

To achieve the pose detection defined by Klein-Vogelbach, 3 classes have to be defined:

- normal posture
- slumped posture
- tensed posture

These classes represent distinct qualitative posture states used in clinical movement analysis. The **normal posture** class represents an aligned and functionally efficient baseline posture. The **slumped posture** class captures flexed, collapsed, or reduced postural tone patterns, typically associated with decreased axial extension. The **tensed posture** class represents increased muscular tone and excessive postural rigidity, often reflected in elevated shoulder girdle and reduced movement fluidity in the spine and hips.

This three-class formulation enables the CNN to distinguish between physiologically neutral, hypoactive (slumped), and hyperactive (tensed) postural states, aligning the computational labels with clinically meaningful categories from the Klein-Vogelbach framework.

5.3.1 Model performance - 3-class problem

For the 3-class problem, in total four training runs were performed using all four available base architectures (from light to heavy computational wise):

- ‘MobileNet-V2‘
- ‘EfficientNet-Lite0‘
- ‘EfficientNet-Lite2‘
- ‘EfficientNet-Lite4‘

For each run the reported metrics below are derived from evaluation on the same held-out on the test split.

5.3.1.1 Training 1 (MobileNet-V2 architecture)

Prediction-derived overall accuracy: 0.3065

Model evaluate(internal performance metric) loss/accuracy on dataset_obj: 0.736599/0.748744

Confusion matrix

True predicted	normal	slumped	tensed
normal	24	21	18
slumped	26	20	16
tensed	23	34	17

Row-normalized confusion matrix (recall by class)

True predicted	normal	slumped	tensed
normal	0.3810	0.3333	0.2857
slumped	0.4194	0.3226	0.2581
tensed	0.3108	0.4595	0.2297

Per-class support:

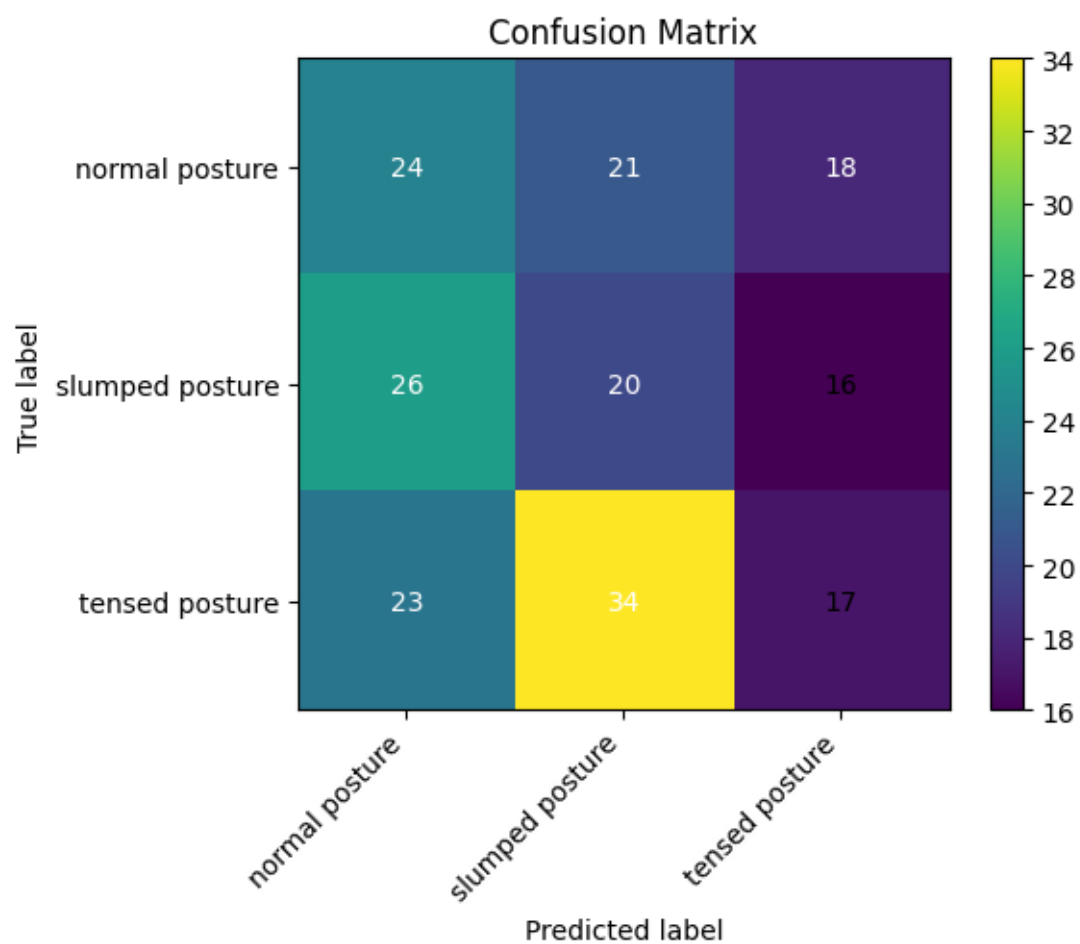
- normal posture: 63
- slumped posture: 62
- tensed posture: 74

Balanced accuracy (macro recall): **0.3111**

Classification report

Class	Precision	Recall	F1-score	Support
normal posture	0.3288	0.3810	0.3529	63
slumped posture	0.2667	0.3226	0.2920	62
tensed posture	0.3333	0.2297	0.2720	74
Accuracy			0.3065	199
Macro avg.	0.3096	0.3111	0.3056	199
Weighted avg.	0.3111	0.3065	0.3038	199

Figure 8: 3-Class - Training 1 confusion matrix showing under-representation of ‘tensed posture’ class.



5.3.1.2 Training 2 (EfficientNet-Lite0 architecture)

Prediction-derived overall accuracy: 0.3216

Model evaluate(internal performance metric) loss/accuracy on dataset_obj: 0.638240/0.824121

Confusion matrix

True predicted	normal	slumped	tensed
normal	4	38	21
slumped	2	40	20
tensed	3	51	20

Row-normalized confusion matrix (recall by class)

True predicted	normal	slumped	tensed
normal	0.0635	0.6032	0.3333
slumped	0.0323	0.6452	0.3226
tensed	0.0405	0.6892	0.2703

Per-class support:

- normal posture: 63
- slumped posture: 62
- tensed posture: 74

Balanced accuracy (macro recall): **0.3263**

Classification report

Class	Precision	Recall	F1-score	Support
normal posture	0.4444	0.0635	0.1111	63
slumped posture	0.3101	0.6452	0.4188	62
tensed posture	0.3279	0.2703	0.2963	74
Accuracy			0.3216	199
Macro avg.	0.3608	0.3263	0.2754	199
Weighted avg.	0.3592	0.3216	0.2759	199

5.3.1.3 Training 3 (EfficientNet-Lite2 architecture)

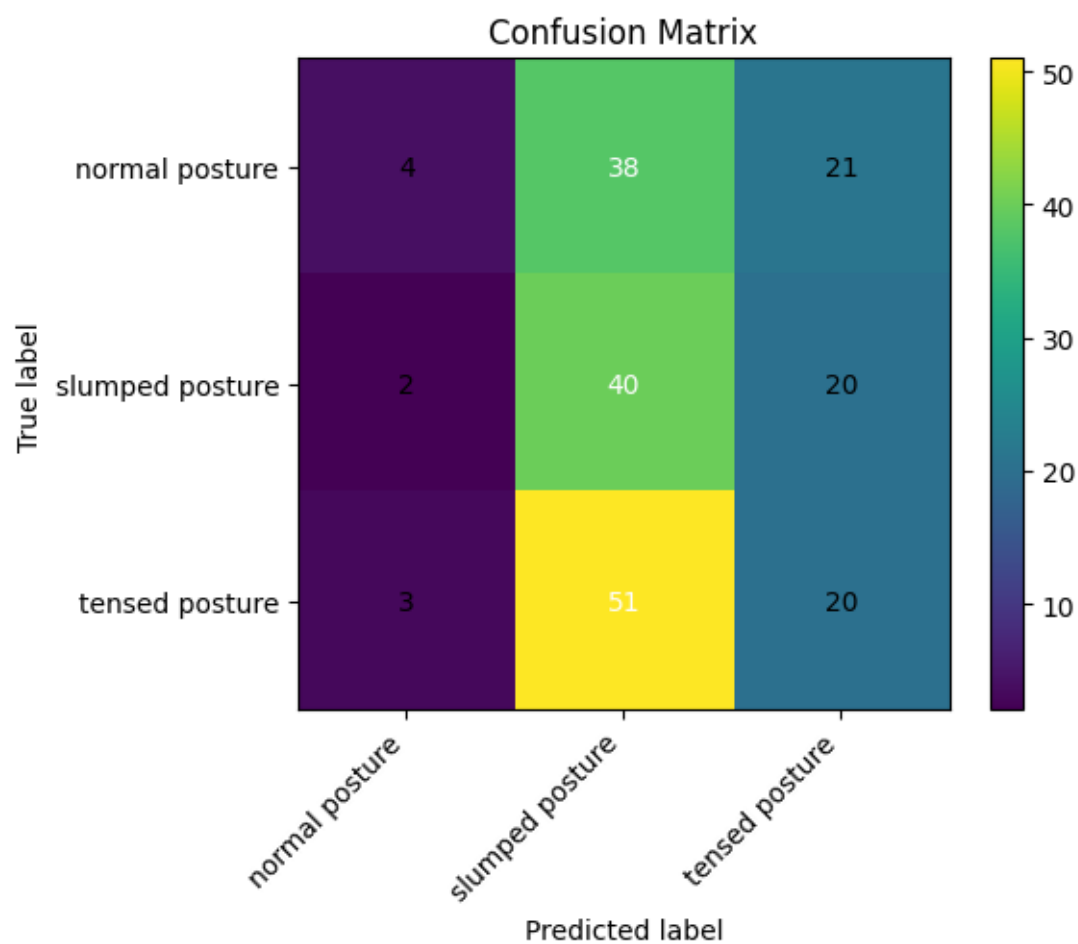
Prediction-derived overall accuracy: 0.3920

Model evaluate(internal performance metric) loss/accuracy on dataset_obj: 0.645087/0.849246

Confusion matrix

True predicted	normal	slumped	tensed
normal	16	36	11
slumped	10	39	13
tensed	12	39	23

Figure 9: 3-Class - Training 2 confusion matrix showing under-representation of ‘normal posture’ class and a preference for ‘slumped posture’.



Row-normalized confusion matrix (recall by class)

True predicted	normal	slumped	tensed
normal	0.2540	0.5714	0.1746
slumped	0.1613	0.6290	0.2097
tensed	0.1622	0.5270	0.3108

Per-class support:

- normal posture: 63
- slumped posture: 62
- tensed posture: 74

Balanced accuracy (macro recall): **0.3979**

Classification report

Class	Precision	Recall	F1-score	Support
normal posture	0.4211	0.2540	0.3168	63
slumped posture	0.3421	0.6290	0.4432	62
tensed posture	0.4894	0.3108	0.3802	74
Accuracy			0.3920	199
Macro avg.	0.4175	0.3979	0.3801	199
Weighted avg.	0.4219	0.3920	0.3797	199

5.3.1.4 Training 4 (EfficientNet-Lite4 architecture)

Prediction-derived overall accuracy: 0.4121

Model evaluate(internal performance metric) loss/accuracy on dataset__obj: 0.703537/0.758794

Confusion matrix

True predicted	normal	slumped	tensed
normal	18	33	12
slumped	12	38	12
tensed	15	33	26

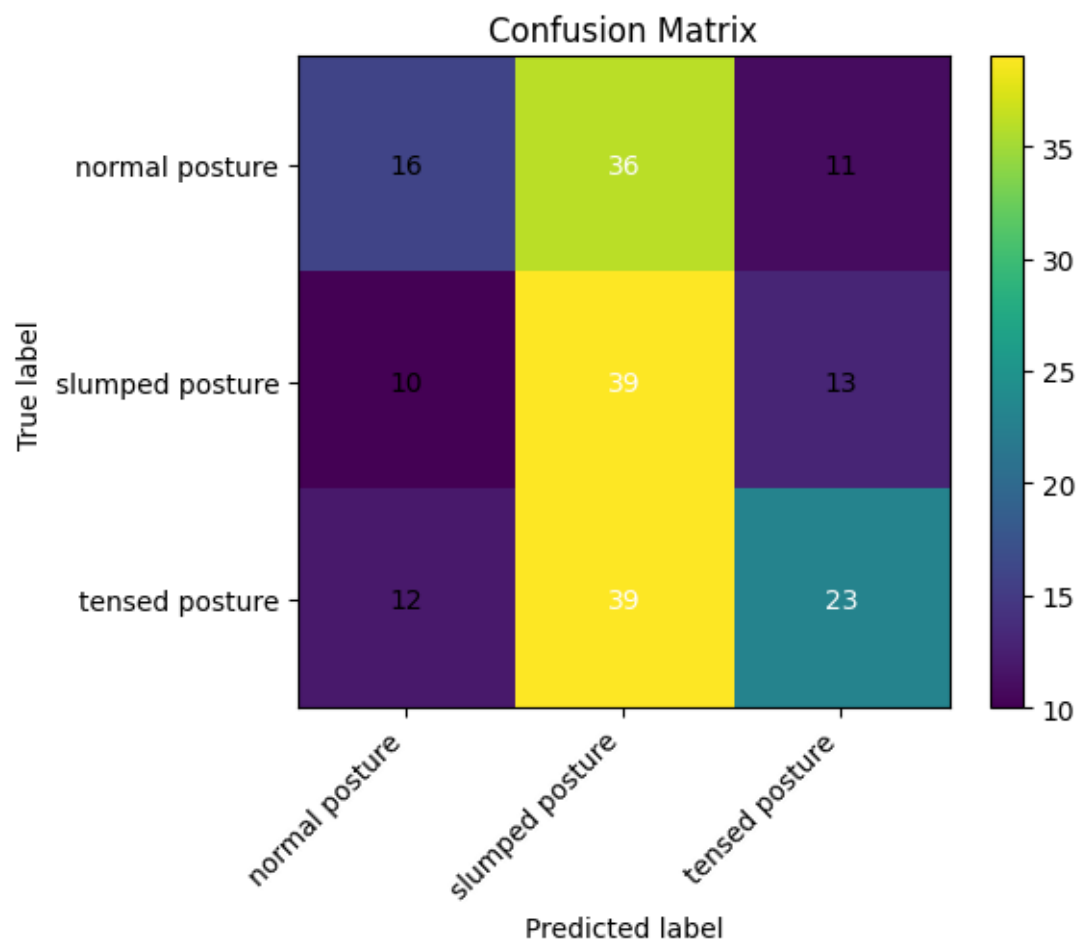
Row-normalized confusion matrix (recall by class)

True predicted	normal	slumped	tensed
normal	0.2857	0.5238	0.1905
slumped	0.1935	0.6129	0.1935
tensed	0.2027	0.4459	0.3514

Per-class support:

- normal posture: 63

Figure 10: 3-Class - Training 3 confusion matrix showing preference of 'slumped posture' class.



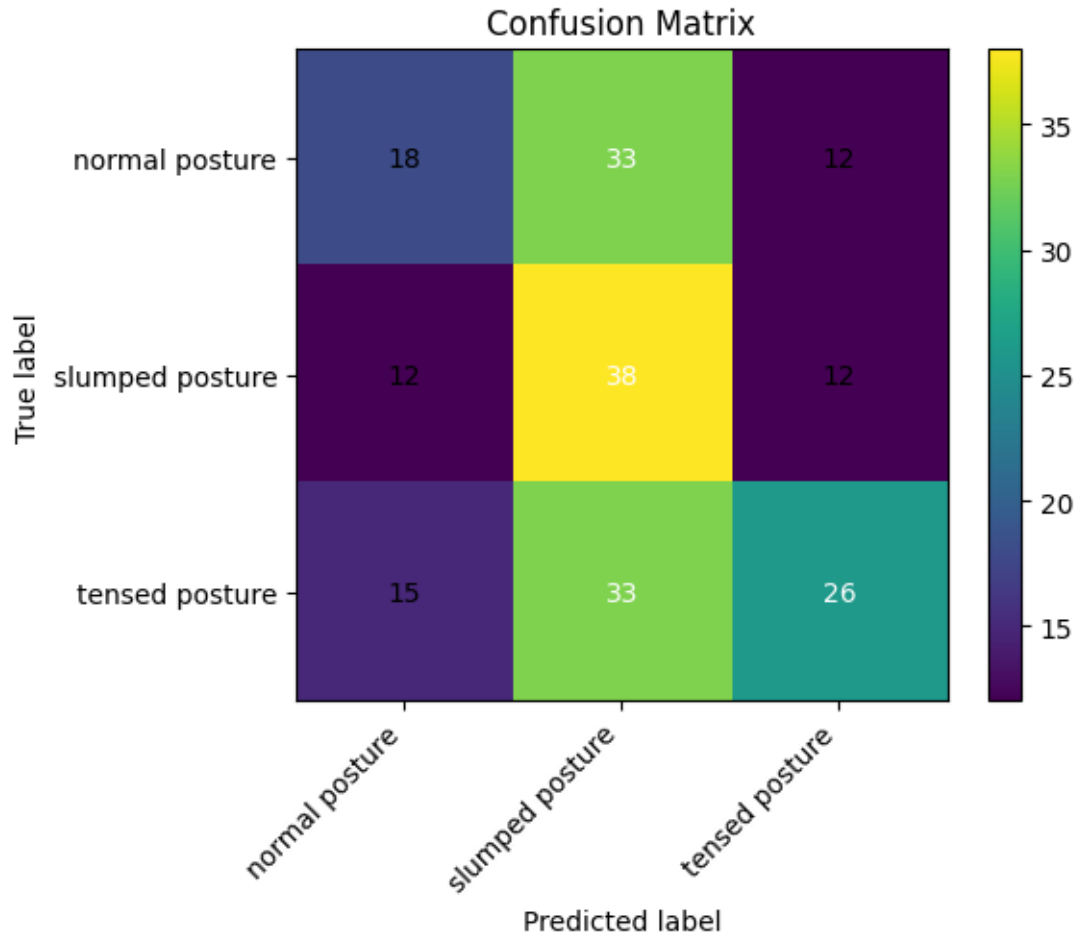
- slumped posture: 62
- tensed posture: 74

Balanced accuracy (macro recall): **0.4167**

Classification report

Class	Precision	Recall	F1-score	Support
normal posture	0.4000	0.2857	0.3333	63
slumped posture	0.3654	0.6129	0.4578	62
tensed posture	0.5200	0.3514	0.4194	74
Accuracy			0.4121	199
Macro avg.	0.4285	0.4167	0.4035	199
Weighted avg.	0.4338	0.4121	0.4041	199

Figure 11: 3-Class - Training 4 confusion matrix showing again preference of ‘slumped posture’ class.



5.3.2 Error analysis

Across all experiments a consistent pattern of misclassification is apparent. The models show relatively high recall for the **slumped posture** class but lower precision, indicating a tendency

to predict **slumped** for a wide range of inputs exemplified visible in Figure 10. **Normal posture** is frequently misclassified (low recall in several runs), and **tensed posture** predictions vary by architecture: deeper Lite variants tended to improve overall discrimination.

- Best overall accuracy and balanced recall were achieved by EfficientNet-Lite4 (accuracy 0.4121, balanced accuracy **0.4167**).
- EfficientNet-Lite2 performed second best (accuracy 0.3920, balanced accuracy **0.3979**).
- EfficientNet-Lite0 and MobileNet-V2 produced lower overall accuracy (0.3216 and **0.3065** respectively) and lower balanced accuracy.

5.3.2.1 Common failure modes observed in confusion matrices

- Confusion between **normal** and **slumped**: many **normal** examples are predicted as **slumped**, suggesting the model is sensitive to subtle postural variations or that the dataset contains ambiguous examples near class boundaries.
- Over-prediction of **slumped**: reflected in row-normalized confusion matrices where the middle column (predicted **slumped**) is large across runs.
- Lower discrimination for **tensed** vs. other classes in the smaller architectures, improved modestly in larger EfficientNet-Lite variants.

5.3.2.2 Comparison of results to 2-class training

Chance levels for the 3-class are 1/3 or 33.3%, for the 2-class 1/2 or 50%. Because of this difference, a chance-balanced accuracy will be calculated.

- Best 3-class model: balanced accuracy = 0.4167.
 - $\ast = 0.4167 - 1/3 = 0.0834$
 - $\ast = 1 - 1/3 - 0.4167 = 0.2834$
 - $\ast = 0.0834 + 0.2834 = 0.3667$
 - $\ast = 0.3667 \times 3 = 1.1001$
 - $\ast = 1.1001 - 1 = 0.1001$
 - $\ast = 0.1001 \times 3 = 0.3003$
 - $\ast = 0.3003 + 0.1001 = 0.4004$
 - $\ast = 0.4004 \times 3 = 1.2012$
 - $\ast = 1.2012 - 1 = 0.2012$
 - $\ast = 0.2012 \times 3 = 0.6036$
 - $\ast = 0.6036 - 0.5 = 0.1036$
 - $\ast = 0.1036 \times 3 = 0.3108$
 - $\ast = 0.3108 + 0.1036 = 0.4144$
 - $\ast = 0.4144 \times 3 = 1.2432$
 - $\ast = 1.2432 - 1 = 0.2432$
 - $\ast = 0.2432 \times 3 = 0.7296$
 - $\ast = 0.7296 - 0.5 = 0.2296$
 - $\ast = 0.2296 \times 3 = 0.6888$
 - $\ast = 0.6888 - 0.5 = 0.1888$
 - $\ast = 0.1888 \times 3 = 0.5664$
 - $\ast = 0.5664 - 0.5 = 0.0664$
 - $\ast = 0.0664 \times 3 = 0.1992$
 - $\ast = 0.1992 + 0.0664 = 0.2656$
 - $\ast = 0.2656 \times 3 = 0.7968$
 - $\ast = 0.7968 - 0.5 = 0.2968$
 - $\ast = 0.2968 \times 3 = 0.8904$
 - $\ast = 0.8904 - 0.5 = 0.3904$
 - $\ast = 0.3904 \times 3 = 1.1712$
 - $\ast = 1.1712 - 0.5 = 0.6712$
 - $\ast = 0.6712 \times 3 = 2.0136$
 - $\ast = 2.0136 - 1 = 1.0136$
 - $\ast = 1.0136 \times 3 = 3.0408$
 - $\ast = 3.0408 - 1 = 2.0408$
 - $\ast = 2.0408 \times 3 = 6.1224$
 - $\ast = 6.1224 - 2 = 4.1224$
 - $\ast = 4.1224 \times 3 = 12.3672$
 - $\ast = 12.3672 - 4 = 8.3672$
 - $\ast = 8.3672 \times 3 = 25.1016$
 - $\ast = 25.1016 - 8 = 17.1016$
 - $\ast = 17.1016 \times 3 = 51.3048$
 - $\ast = 51.3048 - 17 = 34.3048$
 - $\ast = 34.3048 \times 3 = 102.9144$
 - $\ast = 102.9144 - 34 = 68.9144$
 - $\ast = 68.9144 \times 3 = 206.7432$
 - $\ast = 206.7432 - 68 = 138.7432$
 - $\ast = 138.7432 \times 3 = 416.2296$
 - $\ast = 416.2296 - 138 = 278.2296$
 - $\ast = 278.2296 \times 3 = 834.6888$
 - $\ast = 834.6888 - 278 = 556.6888$
 - $\ast = 556.6888 \times 3 = 1670.0664$
 - $\ast = 1670.0664 - 556 = 1114.0664$
 - $\ast = 1114.0664 \times 3 = 3342.1992$
 - $\ast = 3342.1992 - 1114 = 2228.1992$
 - $\ast = 2228.1992 \times 3 = 6684.5976$
 - $\ast = 6684.5976 - 2228 = 4456.5976$
 - $\ast = 4456.5976 \times 3 = 13369.7928$
 - $\ast = 13369.7928 - 4456 = 8913.7928$
 - $\ast = 8913.7928 \times 3 = 26741.3784$
 - $\ast = 26741.3784 - 8913 = 17828.3784$
 - $\ast = 17828.3784 \times 3 = 53485.1352$
 - $\ast = 53485.1352 - 17828 = 35657.1352$
 - $\ast = 35657.1352 \times 3 = 106971.4056$
 - $\ast = 106971.4056 - 35657 = 71314.4056$
 - $\ast = 71314.4056 \times 3 = 213943.2168$
 - $\ast = 213943.2168 - 71314 = 142629.2168$
 - $\ast = 142629.2168 \times 3 = 427887.6504$
 - $\ast = 427887.6504 - 142629 = 285258.6504$
 - $\ast = 285258.6504 \times 3 = 855775.9512$
 - $\ast = 855775.9512 - 285258 = 570517.9512$
 - $\ast = 570517.9512 \times 3 = 1711553.8536$
 - $\ast = 1711553.8536 - 570517 = 1141036.8536$
 - $\ast = 1141036.8536 \times 3 = 3423110.5608$
 - $\ast = 3423110.5608 - 1141036 = 2282074.5608$
 - $\ast = 2282074.5608 \times 3 = 6846223.6824$
 - $\ast = 6846223.6824 - 2282074 = 4564149.6824$
 - $\ast = 4564149.6824 \times 3 = 13692449.0472$
 - $\ast = 13692449.0472 - 4564149 = 9128300.0472$
 - $\ast = 9128300.0472 \times 3 = 27384900.1416$
 - $\ast = 27384900.1416 - 9128300 = 18256600.1416$
 - $\ast = 18256600.1416 \times 3 = 54769800.4248$
 - $\ast = 54769800.4248 - 18256600 = 36513200.4248$
 - $\ast = 36513200.4248 \times 3 = 109539601.2744$
 - $\ast = 109539601.2744 - 36513200 = 73026401.2744$
 - $\ast = 73026401.2744 \times 3 = 219079203.8232$
 - $\ast = 219079203.8232 - 73026401 = 146052802.8232$
 - $\ast = 146052802.8232 \times 3 = 438158408.4696$
 - $\ast = 438158408.4696 - 146052802 = 292105606.4696$
 - $\ast = 292105606.4696 \times 3 = 876316819.4088$
 - $\ast = 876316819.4088 - 292105606 = 584211213.4088$
 - $\ast = 584211213.4088 \times 3 = 1752633640.2264$
 - $\ast = 1752633640.2264 - 584211213 = 1168422427.2264$
 - $\ast = 1168422427.2264 \times 3 = 3505267281.6792$
 - $\ast = 3505267281.6792 - 1168422427 = 2336844854.6792$
 - $\ast = 2336844854.6792 \times 3 = 7010534564.0376$
 - $\ast = 7010534564.0376 - 2336844854 = 4673689710.0376$
 - $\ast = 4673689710.0376 \times 3 = 14021069130.1128$
 - $\ast = 14021069130.1128 - 4673689710 = 9347379420.1128$
 - $\ast = 9347379420.1128 \times 3 = 28042138260.3384$
 - $\ast = 28042138260.3384 - 9347379420 = 18694758840.3384$
 - $\ast = 18694758840.3384 \times 3 = 56084276520.0152$
 - $\ast = 56084276520.0152 - 18694758840 = 37389517680.0152$
 - $\ast = 37389517680.0152 \times 3 = 112168553040.0456$
 - $\ast = 112168553040.0456 - 37389517680 = 74779035360.0456$
 - $\ast = 74779035360.0456 \times 3 = 224337106080.1368$
 - $\ast = 224337106080.1368 - 74779035360 = 149558070720.1368$
 - $\ast = 149558070720.1368 \times 3 = 448674212160.4104$
 - $\ast = 448674212160.4104 - 149558070720 = 299116141440.4104$
 - $\ast = 299116141440.4104 \times 3 = 897348424320.2312$
 - $\ast = 897348424320.2312 - 299116141440 = 598232282880.2312$
 - $\ast = 598232282880.2312 \times 3 = 1794696848640.6936$
 - $\ast = 1794696848640.6936 - 598232282880 = 1196464565760.6936$
 - $\ast = 1196464565760.6936 \times 3 = 3589393697280.5808$
 - $\ast = 3589393697280.5808 - 1196464565760 = 2392929131520.5808$
 - $\ast = 2392929131520.5808 \times 3 = 7178787394560.7424$
 - $\ast = 7178787394560.7424 - 2392929131520 = 4785858263040.7424$
 - $\ast = 4785858263040.7424 \times 3 = 14357574789120.2272$
 - $\ast = 14357574789120.2272 - 4785858263040 = 9571716526080.2272$
 - $\ast = 9571716526080.2272 \times 3 = 28715149578240.6816$
 - $\ast = 28715149578240.6816 - 9571716526080 = 19143433052160.6816$
 - $\ast = 19143433052160.6816 \times 3 = 57430299156480.0448$
 - $\ast = 57430299156480.0448 - 19143433052160 = 38286866104320.0448$
 - $\ast = 38286866104320.0448 \times 3 = 114860598312960.1344$
 - $\ast = 114860598312960.1344 - 38286866104320 = 76573732208640.1344$
 - $\ast = 76573732208640.1344 \times 3 = 229721196625920.4032$
 - $\ast = 229721196625920.4032 - 76573732208640 = 153147464417280.4032$
 - $\ast = 153147464417280.4032 \times 3 = 459442393251840.2096$
 - $\ast = 459442393251840.2096 - 153147464417280 = 306294928834560.2096$
 - $\ast = 306294928834560.2096 \times 3 = 918884786503680.6288$
 - $\ast = 918884786503680.6288 - 306294928834560 = 612589857669120.6288$
 - $\ast = 612589857669120.6288 \times 3 = 1837769573007360.3864$
 - $\ast = 1837769573007360.3864 - 612589857669120 = 1225179715338240.3864$
 - $\ast = 1225179715338240.3864 \times 3 = 3675539146014720.1592$
 - $\ast = 3675539146014720.1592 - 1225179715338240 = 2450359430676480.1592$
 - $\ast = 2450359430676480.1592 \times 3 = 7351078292029440.4776$
 - $\ast = 7351078292029440.4776 - 2450359430676480 = 4900718861352960.4776$
 - $\ast = 4900718861352960.4776 \times 3 = 14702156584058880.4328$
 - $\ast = 14702156584058880.4328 - 4900718861352960 = 9801437722705920.4328$
 - $\ast = 9801437722705920.4328 \times 3 = 29404313168117760.2984$
 - $\ast = 29404313168117760.2984 - 9801437722705920 = 19602875445411840.2984$
 - $\ast = 19602875445411840.2984 \times 3 = 58808626336235520.8952$
 - $\ast = 58808626336235520.8952 - 19602875445411840 = 39205750890823680.8952$
 - $\ast = 39205750890823680.8952 \times 3 = 117617252672471040.2866$
 - $\ast = 117617252672471040.2866 - 39205750890823680 = 78411501781647360.2866$
 - $\ast = 78411501781647360.2866 \times 3 = 235234505344942080.8598$
 - $\ast = 235234505344942080.8598 - 78411501781647360 = 156822993563294720.8598$
 - $\ast = 156822993563294720.8598 \times 3 = 470468980689884160.2730$
 - $\ast = 470468980689884160.2730 - 156822993563294720 = 313645987126589440.2730$
 - $\ast = 313645987126589440.2730 \times 3 = 940937961379768320.8190$
 - $\ast = 940937961379768320.8190 - 313645987126589440 = 627291974253178880.8190$
 - $\ast = 627291974253178880.8190 \times 3 = 1881875922759536640.2557$
 - $\ast = 1881875922759536640.2557 - 627291974253178880 = 1254583948506357760.2557$
 - $\ast = 1254583948506357760.2557 \times 3 = 3763751845519073280.7671$
 - $\ast = 3763751845519073280.7671 - 1254583948506357760 = 2509167897012715520.7671$
 - $\ast = 2509167897012715520.7671 \times 3 = 7527503691038146560.3010$
 - $\ast = 7527503691038146560.3010 - 2509167897012715520 = 5018335794025431040.3010$
 - $\ast = 5018335794025431040.3010 \times 3 = 15055007382076293120.9030$
 - $\ast = 15055007382076293120.9030 - 5018335794025431040 = 10036671588050862080.9030$
 - $\ast = 10036671588050862080.9030 \times 3 = 30110014764152586240.7090$
 - $\ast = 30110014764152586240.7090 - 10036671588050862080 = 20073343176101724160.7090$
 - $\ast = 20073343176101724160.7090 \times 3 = 60220029528305172480.2170$
 - $\ast = 60220029528305172480.2170 - 20073343176101724160 = 40146686352203448320.2170$
 - $\ast = 40146686352203448320.2170 \times 3 = 120440059056610344960.6510$
 - $\ast = 120440059056610344960.6510 - 40146686352203448320 = 80293372704406896640.6510$
 - $\ast = 80293372704406896640.6510 \times 3 = 240880118113220689920.9530$
 - $\ast = 240880118113220689920.9530 - 80293372704406896640 = 160586745408813793280.9530$
 - $\ast = 160586745408813793280.9530 \times 3 = 481760236226441379840.2639$
 - $\ast = 481760236226441379840.2639 - 160586745408813793280 = 321173490817627586560.2639$
 - $\ast = 321173490817627586560.2639 \times 3 = 963520472452882759680.7917$
 - $\ast = 963520472452882759680.7917 - 321173490817627586560 = 642346981635255173120.7917$
 - $\ast = 642346981635255173120.7917 \times 3 = 1927040944905765519360.3751$
 - $\ast = 1927040944905765519360.3751 - 642346981635255173120 = 1284693963270510346240.3751$
 - $\ast = 1284693963270510346240.3751 \times 3 = 3854081889811531038720.1250$
 - $\ast = 3854081889811531038720.1250 - 1284693963270510346240 = 2569387926541020692480.1250$
 - $\ast = 2569387926541020692480.1250 \times 3 = 7708163779623062077440.3750$
 - $\ast = 7708163779623062077440.3750 - 2569387926541020692480 = 5138775853082041384960.3750$
 - $\ast = 5138775853082041384960.3750 \times 3 = 15416327559246124154880.1250$
 - $\ast = 15416327559246124154880.1250 - 5138775853082041384960 = 10277551706164082769920.1250$
 - $\ast = 10277551706164082769920.1250 \times 3 = 30832655118492248309760.3750$
 - $\ast = 30832655118492248309760.3750 - 10277551706164082769920 = 20555103412328165539840.3750$
 - $\ast = 20555103412328165539840.3750 \times 3 = 61665310236984496619520.1250$
 - $\ast = 61665310236984496619520.1250 - 20555103412328165539840 = 41110206824656331079680.1250$
 - $\ast = 41110206824656331079680.1250 \times 3 = 123330620473968993239040.3750$
 - $\ast = 123330620473968993239040.3750 - 41110206824656331079680 = 82220413649312662159360.3750$
 - $\ast = 82220413649312662159360.3750 \times 3 = 246661240947937986478080.1250$
 - $\ast = 246661240947937986478080.1250 - 82220413649312662159360 = 164440827298625324318720.1250$
 - $\ast = 164440827298625324318720.1250 \times 3 = 493322481895875972956160.3750$
 - $\ast = 493322481895875972956160.3750 - 164440827298625324318720 = 328881654597250648637440.3750$
 - $\ast = 328881654597250648637440.3750 \times 3 = 986644963791751945912320.1250$
 - $\ast = 986644963791751945912320.1250 - 328881654597250648637440 = 657763309194501297274880.1250$
 - $\ast = 657763309194501297274880.1250 \times 3 = 1973289927583503891824640.3750$
 - $\ast = 1973289927583503891824640.3750 - 657763309194501297274880 = 1315526618389002594549760.3750$
 - $\ast = 1315526618389002594549760.3750 \times 3 = 3946579855167007783649280.1250$
 - $\ast = 3946579855167007783649280.1250 - 1315526618389002594549760 = 2631053236778005189099520.1250$
 - $\ast = 2631053236778005189099520.1250 \times 3 = 7893159710334015567298560.3750$
 - $\ast = 7893159710334015567298560.3750 - 2631053236778005189099520 = 5262106473556010378199040.3750$
 - $\ast = 5262106473556010378199040.3750 \times 3 = 15786319420668031134597120.1250$
 - $\ast = 15786319420668031134597120.1250 - 5262106473556010378199040 = 10524212947112020756398080.1250$
 - $\ast = 10524212947112020756398080.1250 \times 3 = 31572638841336062269194240.3750$
 - $\ast = 31572638841336062269194240.3750 - 10524212947112020756398080 = 21048425894224041512796160.3750$
 - $\ast$

6 Discussion

This study demonstrates that compact CNNs within the MediaPipe/TFLite stack can learn posture-relevant features, but current discrimination is limited.

6.1 Interpretation of findings

The strongest three-class model (EfficientNet-Lite4) reached 41.2% accuracy (41.7% balanced accuracy). Reformulating the task as binary improved raw accuracy to 55.9% (55.6% balanced) with landmark overlays, indicating that explicit skeletal cues modestly aid detection. To compare the results of the 3-class and 2-class training, we can look at the overall accuracy and balanced accuracy (macro recall) for each model. This 3-class model achieved an overall accuracy of 41.21% and a balanced accuracy of 41.67%, while the 2-class model with landmark overlays achieved an overall accuracy of 55.85% and a balanced accuracy of 55.62%. Also the overall low performances of both CNN model training approaches (2-class as well as 3-class only slightly above chance) suggest that the task is challenging and that further improvements in data quality, model architecture, or training strategy may be needed to achieve clinically useful performance.

6.1.1 Implications and next steps

- The results indicate that the models can capture some posture-related features but that class overlap and limited dataset size constrain discriminative performance. Better class balance, additional labelled data for edge cases, and further augmentation tailored to posture variations should be considered.
- Model calibration and threshold tuning (or use of ensemble methods) may improve precision for the **slumped** and **tensed** classes.
- Further work should include qualitative review of misclassified examples to identify systematic annotation or dataset issues and targeted augmentation or re-labelling where necessary.

6.2 Strengths and limitations

The strongest three-class model (EfficientNet-Lite4) reached 41.2% accuracy (41.7% balanced accuracy). Reformulating the task as binary improved raw accuracy to 55.9% (55.6% balanced) with landmark overlays, indicating that explicit skeletal cues modestly aid detection. However, because the binary chance level is higher (1/2 vs 1/3), these gains largely reflect problem simplification; after accounting for chance, both formulations perform only slightly above baseline. Persistent confusions—especially “normal” misclassified as “slumped”—suggest real class overlap, subtle posture cues, and probable label noise. Given the small dataset and static imagery, these findings point to feasibility rather than readiness for clinical deployment. Priorities for improvement include targeted data growth at class boundaries, refined and possibly multi-rater labels, posture-aware augmentation and sampling, and post-hoc calibration or thresholds to balance sensitivity and specificity for clinically meaningful use.

6.3 Clinical and practical implications

Given current performance (balanced accuracy 41% for three-class), the model’s most appropriate near-term role is as a clinician-in-the-loop decision support tool rather than a standalone diagnostic. Keeping in mind that $n=300$, further annotation and data expansion are needed to train the model to a better overall performance. In practice, the system can assist screening by flagging potentially “non-normal” cases for closer review, standardize documentation via landmark/angle overlays, and track an individual’s progress over time under consistent capture conditions. Safe deployment requires confidence calibration with an “abstain/needs review” state, threshold tuning to the intended use (e.g., sensitivity-oriented screening), and clear presentation of uncertainty. To enhance reliability and equity, future work should expand and re-balance labeled data—particularly at class boundaries—adopt multi-rater labeling, incorporate interpretable posture-aware features (e.g., craniovertebral angle), and validate externally across sites, devices, and demographic subgroups. On-device TFLite inference supports privacy and low latency, but any clinical use should be governed by explicit intended-use documentation, bias and robustness monitoring.

7 Conclusion and Future Work

This dissertation demonstrates that transfer learning on compact CNNs within the MediaPipe/TFLite stack is feasible for static upper-body posture assessment, but current performance does not yet meet clinical decision thresholds. The strongest three-class model (EfficientNet-Lite4) achieved 41.2% accuracy (41.7% balanced accuracy). Persistent confusions—particularly over-prediction of “slumped” and low recall for “normal”—likely reflect subtle class boundaries, label noise, and the small, static dataset. Practically, the present system is best positioned as clinician-in-the-loop decision support for screening, standardized documentation (landmark/angle overlays), and within-subject progress tracking under a consistent capture protocol. On-device TFLite supports privacy and low latency, but safe use requires calibrated confidence, explicit uncertainty, and clear guardrails around intended use.

Priorities for future work will center on prospective clinical image collection, expert annotation, improving the MediaPipe Pose detection, develop a 4-body-block detection model (Klein-Vogelbach) as well as retraining the three-class model to improve discrimination and equity.

7.1 Clinical data collection

To acquire a larger, prospectively collected side-view image dataset in real clinical settings, a strictly standardized acquisition protocol should be implemented, including fixed camera height at approximately shoulder level, constant camera-to-subject distance, standardized lateral stance, neutral footwear, uncluttered background, and controlled, even lighting conditions. Demographic representativeness across relevant population groups should be ensured as well informed consent and ethics approvals should be obtained. In addition, metadata such as device type, viewing angle, and lighting conditions could be systematically recorded to enable reproducibility and avoid bias assessment and ensuring robust model validation.

7.2 Expert annotation

For expert annotation, future work should implement a multi-rater labeling protocol with at least two independent clinicians to ensure annotation reliability, in contrast to the present study where annotations were performed by a single rater (the author). All annotations should follow a clearly defined rubric aligned with the Klein-Vogelbach posture classes (normal, slumped/hypotonic, tensed/hypertonic), including explicit decision criteria to reduce subjective variability. Disagreements between raters should be resolved through a structured adjudication process, ideally involving a third expert reviewer.

Inter-rater agreement should be quantified using metrics such as Cohen’s κ to monitor consistency and identify ambiguous cases. In addition, a stable **gold-standard** subset should be maintained and periodically reused for model calibration and regression testing to ensure annotation quality and temporal consistency across dataset iterations.

7.3 Retraining the 3-class model

To retrain the model using class-balanced sampling to mitigate class imbalance and apply posture-aware augmentation, boundary cases between normal, slumped, and tensed postures should be carefully handled rather than inadvertently reinforced. Training should incorporate more discriminative and clinically consistent examples, with targeted augmentation that preserves posture semantics while increasing variability in non-ambiguous cases.

In addition, interpretable posture-derived features extracted from landmarks (e.g., craniovertebral angle and thoracic kyphosis proxies) could be integrated to complement image-based learning with clinically meaningful structure. A multi-task architecture, jointly predicting posture class and continuous postural angles, may further improve both classification performance and interpretability by enforcing consistency between categorical and geometric representations of posture. Finally, probability calibration should be applied, and operating thresholds should be explicitly defined for a screening use case to prioritize sensitivity for non-normal postures while controlling false-positive rates.

7.4 Evaluation and fairness

Future work should include external validation on a held-out site or device to assess generalisability beyond the current data source test data-set. In addition, model performance should be reported across relevant subgroups (e.g., sex, age bands, and skin tone), including an analysis of confusion patterns to identify systematic biases or failure modes. Learning-curve analyses should further be conducted to quantify data requirements and inform targeted data collection for improving model robustness and fairness.

7.5 Deployment readiness

For deployment readiness, inference should be performed on-device to ensure low latency and preserve user privacy within the authors E-Health SaaS service *Serious Ben Entertainment* for the German health market, where posture detection is embedded in a digital serious back school game. To enhance reliability in this preventive context, the system should implement confidence gating, allowing the model to abstain or flag cases as “needs review” when prediction certainty is low. In addition, datasets and model versions should be systematically tracked to ensure reproducibility and regulatory transparency. Post-deployment, continuous monitoring for data drift is required, with periodic recalibration to maintain performance as usage conditions, devices, and user populations evolve.

7.6 Milestones

As a concrete milestone, a on-site pilot study should be conducted creating approximately 1,000 newly class-balanced clinical images. Model performance should be evaluated on an external test split, with a target balanced accuracy (three-class, macro recall) in the range of **0.60–0.65** to demonstrate improved generalisability and clinically meaningful classification performance.

Further future work should prioritize the transition to a structured four-body-block pose detection approach aligned with Klein-Vogelbach’s framework. This involves training models to estimate the relative position and orientation of the head, thorax, pelvis, and lower extremities, enabling direct modelling of inter-segment relationships rather than relying on high-dimensional joint inputs. Such an approach may improve robustness and interpretability. Model development should be accompanied by clinically informed annotation protocols and validated against expert-labelled reference data to ensure alignment with functional movement assessment standards.

Last but not least, a dedicated iOS application should be developed to deploy the trained models on Apple’s A-series chip architecture, leveraging the integrated Neural Engine for efficient on-device inference. This would enable real-time posture analysis with low latency, reduced energy consumption, and enhanced privacy through local processing, while also supporting scalable deployment within mobile E-Health and serious gaming applications.

8 Appendix A. Figures

Figure 12: Google MediaPipe Pose overlay to show angles of head/neck and torso.

Figure 12: Overlay annotation by Google MediaPipe Pose.



References

- Badhe, P. C., & Kulkarni, V. (2018). A review on posture assessment. In <https://www.iosrjournals.org/iosr-jspe/papers/Vol-5Issue5/Version-1/B05050815.pdf>. https://www.researchgate.net/publication/328981577_A_Review_on_Posture_Assessment
- Bazarevsky, V., & Grishchenko, I. (2020). *On-device, real-time body pose tracking with MediaPipe BlazePose*. <https://research.google/blog/on-device-real-time-body-pose-tracking-with-mediapipe-blazepose/>
- Cao, X., Wang, X., Geng, X., Wu, D., & An, H. (2024). An approach for human posture recognition based on the fusion PSE-CNN-BiGRU model. *CMES - Computer Modeling in Engineering and Sciences*, 140, 385–408. <https://doi.org/10.32604/CMES.2024.046752>
- F.M. Alexander. (2017). Methode mit großem betriebsmedizinischem potential eigene, die berufsfähigkeit existentieller bedrohende probleme mit der stimme führten. *Institut Für Arbeits-, Sozial- Und Umweltmedizin*. <https://doi.org/10.2017>
- Ge, L., Pereira, M. J., Yap, C. W., & Heng, B. H. (2022). Chronic low back pain and its impact on physical function, mental health, and health-related quality of life: A cross-sectional study in singapore. *Scientific Reports*, 12. <https://doi.org/10.1038/S41598-022-24703-7>
- Google. (2024). *Pose detection | ML kit | google for developers*. <https://developers.google.com/ml-kit/vision/pose-detection>
- Jiang, X., Hu, Z., Wang, S., & Zhang, Y. (2023). A survey on artificial intelligence in posture recognition. *Computer Modeling in Engineering & Sciences : CMES*, 137, 35. <https://doi.org/10.32604/CMES.2023.027676>
- Kakuta, K. C., Kikawa, T., Kumagai, Y., Gupta, J., Pathak, S., & Kumar, G. (2022). Deep learning (CNN) and transfer learning: A review. *Journal of Physics: Conference Series*, 2273, 012029. <https://doi.org/10.1088/1742-6596/2273/1/012029>
- Klein-Vogelbach, S. (1990). Functional kinetics. *Functional Kinetics*. <https://doi.org/10.1007/978-3-642-95470-2>
- Kubalek-Schröder, S., & Dehler, F. (2013). Funktionsabhängige beschwerdebilder des bewegungssystems. *Funktionsabhängige Beschwerdebilder Des Bewegungssystems*. <https://doi.org/10.1007/978-3-642-35151-8>
- Lachance, J. M., Thong, W., Shruti, N., & Xiang, A. (2023). A case study in fairness evaluation: Current limitations and challenges for human pose estimation. *Sony*. <https://r2hcai.github.io/AAAI-23/files/CameraReadys/21.pdf>
- Liu, M., Rong, J., An, X., Li, Y., Min, Y., Yuan, G., Yang, Y., & Li, M. (2025). Global, regional, and national burden of musculoskeletal disorders, 1990–2021: An analysis of the global burden of disease study 2021 and forecast to 2035. *Frontiers in Public Health*, 13, 1562701. <https://doi.org/10.3389/FPUH.2025.1562701/FULL>
- Lugaresi, C., Tang, J., Nash, H., Mcclanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., Grundmann, M., & Research, G. (2019). MediaPipe: A framework for building perception pipelines. *Google*. <https://arxiv.org/pdf/1906.08172>
- Move AI. (2026). Move AI - markerless motion capture & 3D animation. In <https://move.ai>. <https://move.ai/>
- Ray, A., & Kolekar, M. H. (2024). Transfer learning and its extensive appositeness in human activity recognition: A survey. *Expert Systems with Applications*, 240, 122538. <https://doi.org/10.1016/J.ESWA.2023.122538>
- RunDNA. (2026). Real-time 3D running gait analysis. <https://Rundna.com/>. <https://rundna.com/helix3d/>
- RUNRIGHT 3D. (2026). A running gait analysis and shoe fitting system. In <https://runright-3d.com>. <https://runright-3d.com/>
- Sahrmann, S., Azevedo, D. C., & Dillen, L. V. (2017). Diagnosis and treatment of movement

- system impairment syndromes. *Brazilian Journal of Physical Therapy*, 21, 391. <https://doi.org/10.1016/J.BJPT.2017.08.001>
- Tachihara, H., & Hamada, J. (2019). Characteristic movement of the ribs, thoracic vertebrae while elevating the upper limbs - influences of age and gender on movements. *The Open Orthopaedics Journal*, 13, 170–176. <https://doi.org/10.2174/1874325001913010170>
- TecnoBody Walker View. (2026). TecnoBody view 3.0 SCX treadmill. In <https://www.medical-xprt.com/products/tecnobody-model-view-30-scx-treadmill-804362>. <https://www.medical-xprt.com/products/tecnobody-model-view-30-scx-treadmill-804362>
- Ultralytics. (2025). Pose estimation - ultralytics YOLO docs. In *ultralytics*. <https://docs.ultralytics.com/tasks/pose/>
- ViMove2. (2026). ViMove2: Wearable sensor technology. In <https://runreborn.com/article/vi-move2-wearable-sensor-technology>. <https://runreborn.com/article/vi-move2-wearable-sensor-technology>
- Wagh, V., Scott, M. W., & Kraeutner, S. N. (2024). Quantifying similarities between MediaPipe and a known standard to address issues in tracking 2D upper limb trajectories: Proof of concept study. *JMIR Formative Research*, 8, e56682. <https://doi.org/10.2196/56682>
- Whittaker, J. L., Booyesen, N., Motte, S. D. L., Dennett, L., Lewis, C. L., Wilson, D., McKay, C., Warner, M., Padua, D., Emery, C. A., & Stokes, M. (2017). Predicting sport and occupational lower extremity injury risk through movement quality screening: A systematic review. *British Journal of Sports Medicine*, 51, 580–585. <https://doi.org/10.1136/BJSPORTS-2016-096760>
- Wijekulasuriya, G. A., Woods, C. T., Kittel, A., & Larkin, P. (2025). The development and content of movement quality assessments in athletic populations: A systematic review and multilevel meta-analysis. *Sports Medicine - Open*, 11, 7. <https://doi.org/10.1186/S40798-025-00813-0>
- Xu, H., Bazavan, E. G., Zanzfir, A., Freeman, W. T., Sukthankar, R., & Sminchisescu, C. (2020). GHUM GHUML: Generative 3D human shape and articulated pose models. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 6183–6192. <https://doi.org/10.1109/CVPR42600.2020.00622>
- Zaucker, P. (2010). *Ballkoordination bei schülern mit unterschiedlicher sporterfahrung*.